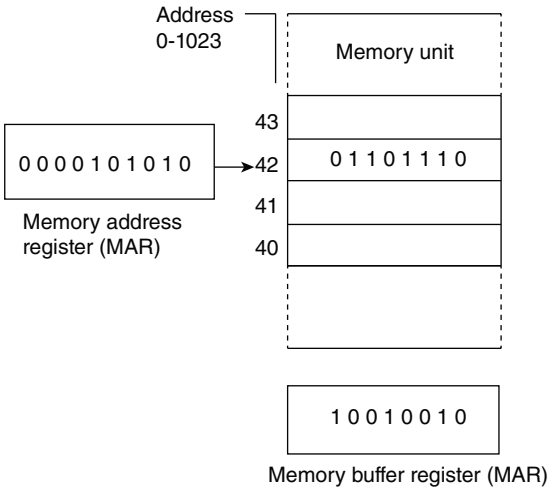


Figure 7-25 Initial values of registers

an address of ten bits, since $2^{10} = 1024$. Therefore, the address register must contain ten flip-flops. The buffer register must have eight flip-flops to store the contents of words transferred into and out of memory. The memory unit has 1024 registers with assigned address numbers from 0 to 1023.

Figure 7-25 shows the initial contents of three registers; memory address register (MAR), memory buffer register (MBR), and the memory register addressed by MAR. Since the equivalent binary number in MAR is decimal 42, the memory register addressed by MAR is the one with address number 42.

The sequence of operations needed to communicate with the memory unit for the purpose of transferring a word out to the MBR is:

1. Transfer the address bits of the selected word into MAR.
2. Activate the *read* control input.

The result of the read operation is depicted in Fig. 7-26(a). The binary information presently stored in memory register 42 is transferred into MBR.

The sequence of operations needed to store a new word into memory is:

1. Transfer the address bits of the selected word into MAR.
2. Transfer the data bits of the word into MBR.
3. Activate the *write* control input.

The result of the write operation is depicted in Fig. 7-26(b), The data bits from MBR are stored in memory register 42.

In the above example, we assumed a memory unit with nondestructive read-out property. Such memories can be constructed with semiconductor ICs. They retain the information in the memory register when the register is sampled during the reading process so that no loss of information

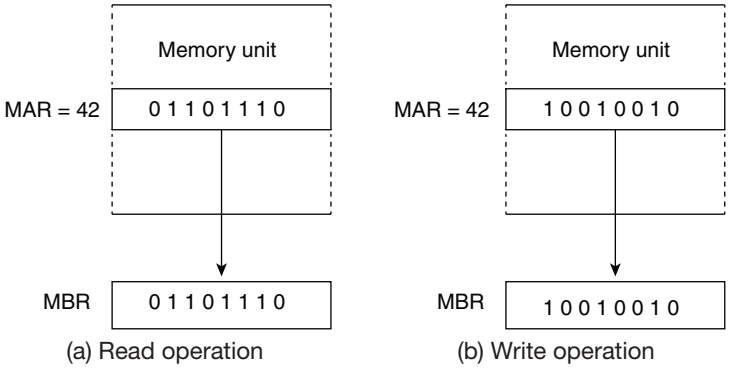


Figure 7-26 Information transfer during read and write operations

occurs. Another component commonly used in memory units is the magnetic core. A magnetic core characteristically has destructive read-out, i.e., it loses the stored binary information during the reading process. Examples of semiconductor and magnetic-core memories are presented in Section 7-8.

Because of its destructive read-out property, a magnetic-core memory must provide additional control functions to restore the word into the memory register.

A read control signal applied to a magnetic-core memory transfers the content of the addressed word into an external register and, at the same time, the memory register is automatically cleared. The sequence of internal control in a magnetic-core memory then provides appropriate signals to cause the restoration of the word into the memory register. The information transfer in a magnetic-core memory during a read operation is depicted in Fig. 7-27. A destructive read operation transfers the selected word into MBR but leaves the memory register with all 0's. Normal memory operation requires that the content of the selected word remain in memory after a read operation. Therefore, it is necessary to go through a *restore* operation that writes the value in MBR into the selected memory register. During the restore operation, the contents of MAR and MBR must remain unchanged.

A write control input applied to a magnetic-core memory causes a transfer of information as depicted in Fig. 7-28. To transfer new information into a selected register, the old information

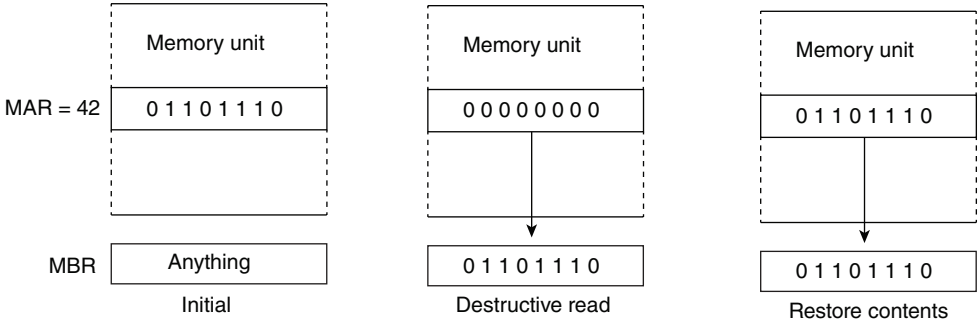


Figure 7-27 Information transfer in a magnetic-core memory during a read operation

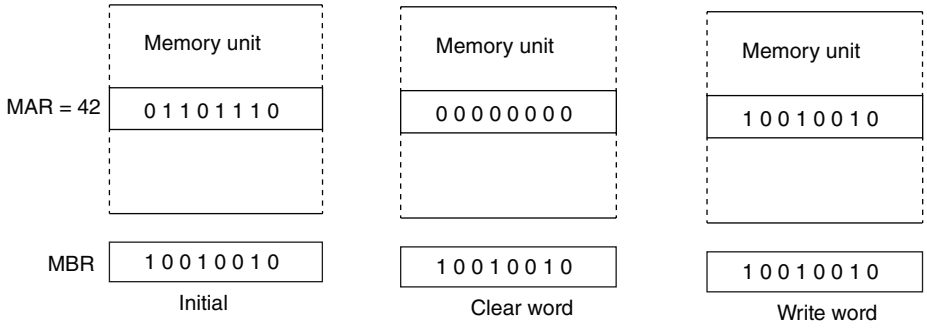


Figure 7-28 Information transfer in a magnetic-core memory during a write operation

must first be erased by clearing all the bits of the word to 0. After this is done, the content of MBR can be transferred to the selected word. MAR must not change during the operation to ensure that the same selected word that is cleared is the one that receives the new information.

A magnetic-core memory requires two half-cycles either for reading or writing. The time it takes for the memory to go through both half-cycles is called the *memory-cycle* time.

The mode of access of a memory system is determined by the type of components used. In a *random-access* memory, the registers may be thought of as being separated in space, with each register occupying one particular spatial location as in a magnetic-core memory. In a *sequential-access* memory, the information stored in some medium is not immediately accessible but is available only at certain intervals of time. A magnetic-tape unit is of this type. Each memory location passes the read and write heads in turn, but information is read out only when the requested word has been reached. The *access time* of a memory is the time required to select a word and either read or write it. In a random-access memory, the access time is always the same regardless of the word's particular location in space. In a sequential memory, the access time depends on the position of the word at the time of request. If the word is just emerging from storage at the time it is requested, the access time is just the time necessary to read or write it. But, if the word happens to be in the last position, the access time also includes the time required for all the other words to move past the terminals. Thus, the access time in a sequential memory is variable.

Memory units whose components lose stored information with time or when the power is turned off are said to be *volatile.* A semiconductor memory unit is of this category since its binary cells need external power to maintain the needed signals. In contrast, a nonvolatile memory unit, such as magnetic core or magnetic disk, retains its stored information after removal of power. This is because the stored information in magnetic components is manifested by the direction of magnetization, which is retained when power is turned off. A nonvolatile property is desirable in digital computers because many useful programs are left permanently in the memory unit. When power is turned off and then on again, the previously stored programs and other information are not lost but continue to reside in memory.

7.8 Examples of Random-access Memories

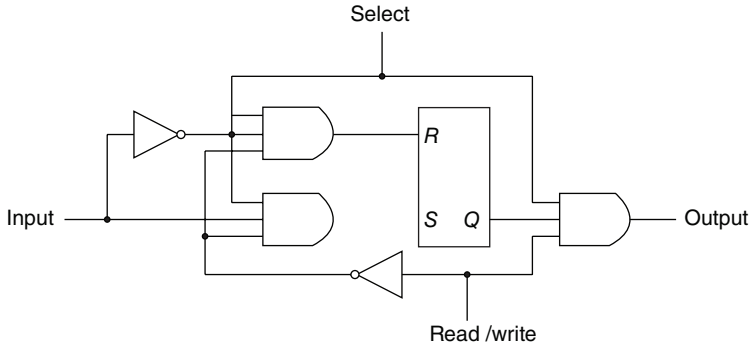
The internal construction of two different types of random-access memories are presented diagramatically in this section. The first is constructed with flip-flops and gates and the second with magnetic cores. To be able to include the entire memory unit in one diagram, a limited storage

capacity must be used. For this reason, the memory units presented here have a small capacity of 12 bits arranged in four words of three bits each. Commercial random-access memories may have a capacity of thousands of words and each word may range somewhere between 8 and 64 bits. The logical construction of large-capacity memory units would be a direct extension of the configuration shown here.

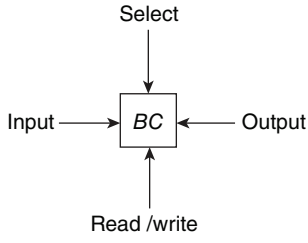
7.8.1 Integrated-circuit Memory

The internal construction of a random-access memory of m words with n bits per word consists of $m \times n$ binary storage cells and the associated logic for selecting individual words. The binary storage cell is the basic building block of a memory unit. The equivalent logic of a binary cell that stores one bit of information is shown in Fig. 7-29. Although the cell is shown to include gates and a flip-flop, internally it is constructed with two transistors having multiple inputs. A binary storage cell must be very small in order to be able to pack as many cells as possible in the small area available in the integrated-circuit chip. The binary cell has three inputs and one output. The select input enables the cell for reading or writing. The read/write input determines the cell operation when it is selected. A 1 in the read/write input forms a path from the flip-flop to the output terminal. The information in the input terminal is transferred into the flip-flop when the read/write control is 0. Note that the flip-flop operates without clock pulses and that its purpose is to store the information bit in the binary cell.

Integrated-circuit memories sometimes have a single line for the read and write control. One binary state in the single line specifies a read operation and the other state specifies a write



(a) Logic diagram



(b) Block diagram

Figure 7-29 Memory cell

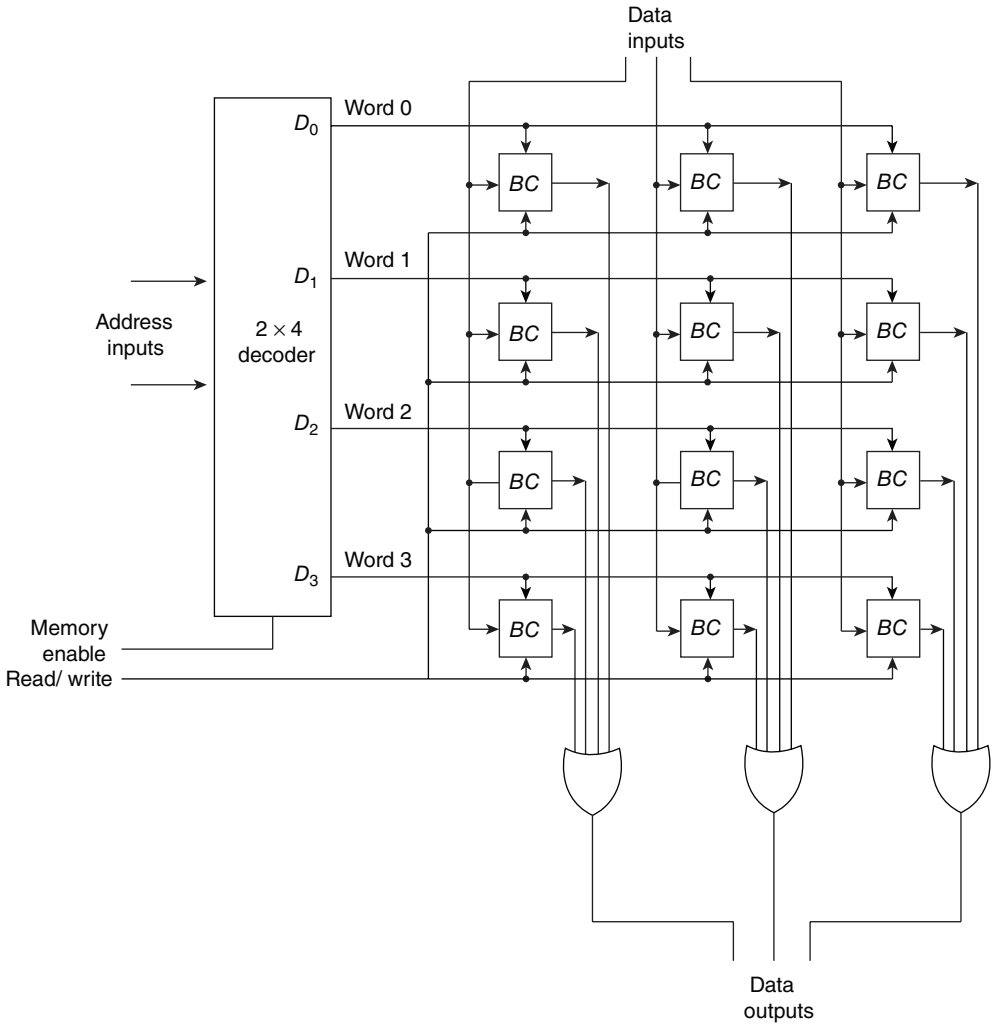


Figure 7-30 Integrated-circuit memory

operation. In addition, one or more enable lines are included to provide means for selecting the IC and for expanding several packages into a memory unit with a larger number of words. The logical construction of an IC RAM is shown in Fig. 7-30. It consists of 4 words of 3 bits each, for a total of 12 binary cells. The small boxes labeled BC represent a binary cell, and the three inputs and one output in each BC are as specified in the diagram of Fig. 7-29.

The two address input lines go through an internal 2-to-4 line decoder. The decoder is enabled with the memory-enable input. When the memory enable is 0, all the outputs of the decoder are 0 and none of the memory words are selected. With the memory enable at 1, one of the four words is selected, depending on the value of the two address lines. Now, with the read/write control at 1, the bits of the selected word go through the three OR gates to the output terminals. The nonselected binary cells produce 0's in the inputs of the OR gates and have no effect on the out-

puts. With the read/write control at 0, the information available on the input lines is transferred into the binary cells of the selected word. The nonselected binary cells in the other words are disabled by their selection inputs and their previous values remain unchanged. With the memory-enable control at 0, the contents of all cells in the memory remain unchanged, regardless of the value of the read/write control.

IC RAMs are constructed internally with cells having a wired-OR capability. This eliminates the need for the OR gates in the diagram. The external output lines can also form wired logic to facilitate the connection of two or more IC packages to form a memory unit with a larger number of words.

7.8.2 Magnetic-core Memory

A magnetic-core memory uses magnetic cores to store binary information. A magnetic core is a doughnut-shaped toroid made of magnetic material. In contrast to a semiconductor flip-flop that needs only one physical quantity such as voltage for its operation, a magnetic core employs three physical quantities; current, magnetic flux, and voltage. The signal that excites the core is a *current* pulse in a wire passing through the core. The binary information stored is represented by the direction of *magnetic flux* within the core. The output binary information is extracted from a wire linking the core in the form of a *voltage* pulse.

The physical property that makes a magnetic core suitable for binary storage is its hysteresis loop, shown in Fig. 7-31(c). This is a plot of current vs magnetic flux, and it has the shape of a square loop. With zero current, a flux which is either positive (counter clockwise direction) or negative (clockwise direction) remains in the magnetized core. One direction, say counter clockwise magnetization, is used to represent a 1 and the other to represent a 0.

A pulse of current applied to the winding through the core can shift the direction of magnetization. As shown in Fig. 7-31(a), current in the downward direction produces flux in the clockwise direction, causing the core to go to the 0 state. Figure 7-31(b) shows the current and flux directions for storing a 1. The path that the flux takes when the current pulse is applied is indicated by arrows in the hysteresis loop.

Reading out the binary information stored in the core is complicated by the fact that flux cannot be detected when it is not changing. However, if flux is changing with respect to time, it induces a voltage in a wire that links the core. Thus, read-out could be accomplished by applying a current in the negative direction as shown in Fig. 7-32. If the core is in the 1 state, the current

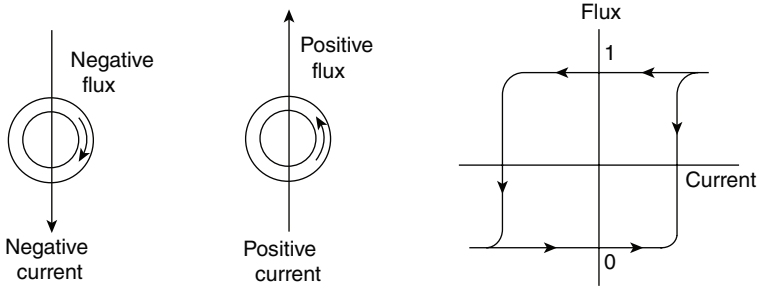


Figure 7-31 Storing a bit into a magnetic core

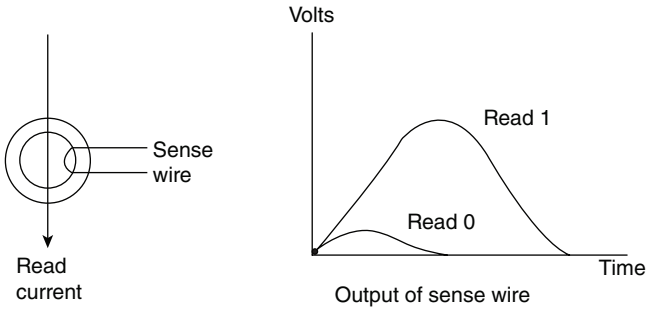


Figure 7-32 Reading a bit from a magnetic core

reverses the direction of magnetization, and the resulting change of flux produces a voltage pulse in the sense wire. If the core is already in the 0 state, the negative current leaves the core magnetized in the same direction, causing a very slight disturbance of magnetic flux which results in a very small output voltage in the sense wire. Note that this is a destructive read-out, since the read current always returns the core to the 0 state. The previously stored value is lost.

Figure 7-33 shows the organization of *S* magnetic-core memory containing four words with three bits each. Comparing it with the IC memory unit of Fig. 7-30, we note that the binary cell now is a magnetic core and the wires linking it. The excitation of the core is accomplished by means of a current pulse generated in a driver (DR). The output information goes through a sense amplifier (SA) whose outputs set corresponding flip-flops in the buffer register. Three wires link each core. The word wire is excited by a word driver and goes through the three cores of a word. A bit wire is excited by a bit driver and goes through four cores in the same bit position. The sense wire links the same cores as the bit wire and is applied to a sense amplifier that shapes the voltage pulse when a 1 is read and rejects the small disturbance when a 0 is read.

During a read operation, a word-driver current pulse is applied to the cores of the word selected by the decoder. The read current is in the negative direction (Fig. 7-32) and causes all cores of the selected word to go to the 0 state, regardless of their previous state. Cores which previously contained a 1 switch their flux and induce a voltage into their sense wire. The flux of cores which already contained a 0 is not changed. The voltage pulse on a sense wire of cores with a previous 1 is amplified in the sense amplifier and sets the corresponding flip-flop in the buffer register.

During a write operation, the buffer register holds the information to be stored in the word specified by the address register. We assume that all cores in the selected word are initially cleared, i.e., all are in the 0 state so that cores requiring a 1 need to undergo a change of state. A current pulse is generated simultaneously in the word driver selected by the decoder and in the bit driver, whose corresponding buffer register flip-flop contains a 1. Both currents are in the positive direction, but their magnitude is only half that needed to switch the flux to the 1 state. This half-current by itself is too small to change the direction of magnetization. But the sum of two half-currents is enough to switch the direction of magnetization to the 1 state. A core switches to the 1 state only if there is a coincidence of two half-currents from a word driver and a bit driver. The direction of magnetization of a core does not change if it receives only half-current from one of the drivers. The result is that the magnetization of cores is switched to the 1 state only if the word and bit wires intersect, that is only in the selected word and only in the bit position in which the buffer register is a 1.

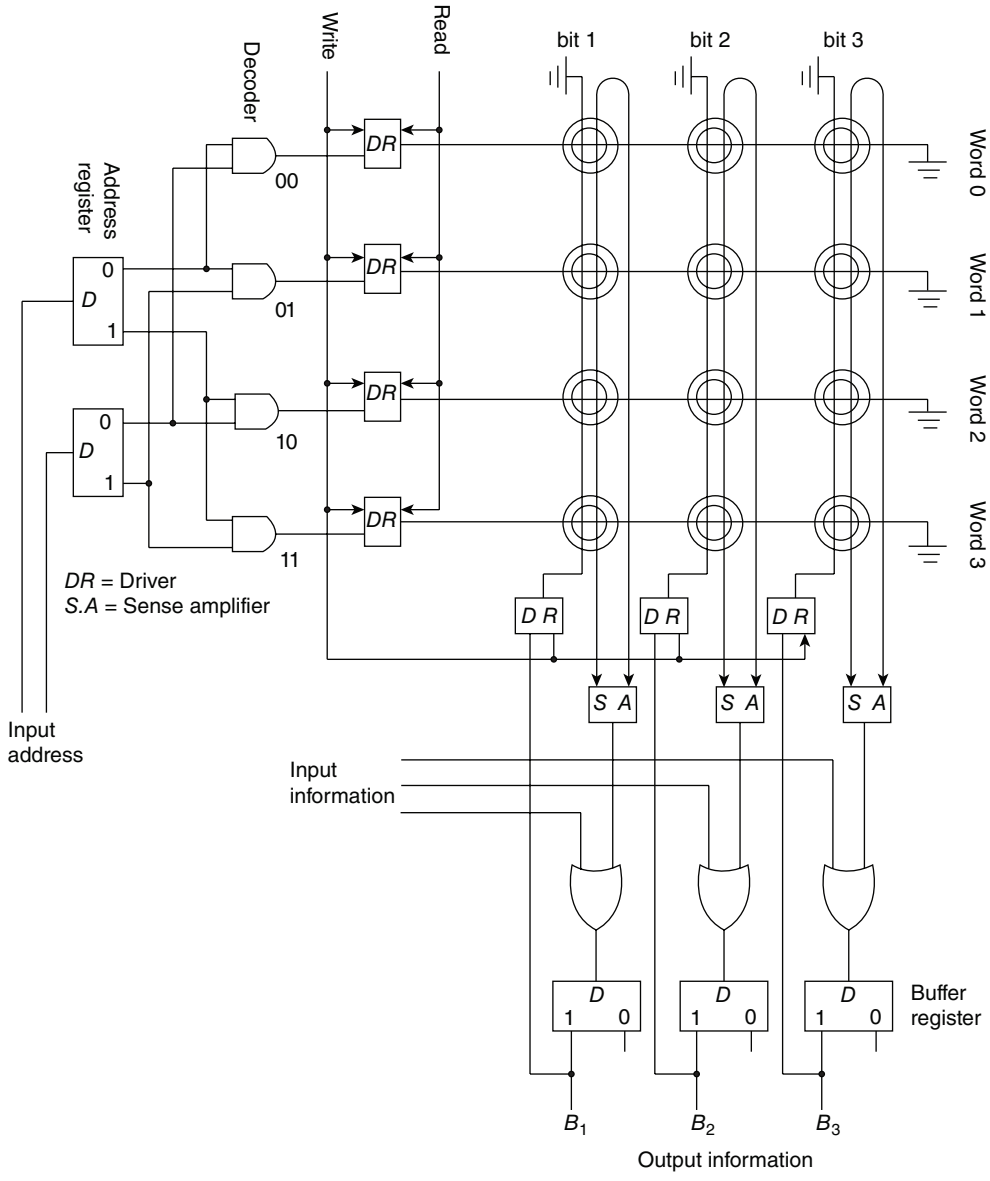


Figure 7-33 Magnetic-core memory unit

The read and write operations described above are incomplete, because the information stored in the selected word is destroyed by the reading process and the write operation works properly only if the cores are initially cleared. As mentioned in Section 7-7, a read operation must be followed by another cycle that restores the values previously stored in the cores. A write operation is preceded by a cycle that clears the cores of the selected word.

The restore operation during a read cycle is equivalent to a write operation which, in effect, writes the previously read information from the buffer register back into the word selected. The clear operation during a write cycle is equivalent to a read operation which destroys the stored information but prevents the read information from reaching the buffer register by inhibiting the sense amplifier. Restore and clear cycles are normally initiated by the memory internal control, so that the memory unit appears to the outside as having a nondestructive read-out property.

REFERENCES

1. *The TTL Data Book for Design Engineers.* Dallas, Texas: Texas Instruments, Inc., 1976.
2. Blakeslee, T. R., *Digital Design with Standard MSI and LSI.* New York: John Wiley & Sons, 1975.
3. Barna A., and D. I. Porat, *Integrated Circuits in Digital Electronics.* New York: John Wiley & Sons, 1973.
4. Taub, H., and D. Schilling, *Digital Integrated Electronics.* New York: McGraw-Hill Book Co., 1977.
5. Grinich, V. H., and H. G. Jackson, *Introduction to Integrated Electronics.* New York: McGraw-Hill Book Co., 1975.
6. Kostopoulos, G. K., *Digital Engineering.* New York: McGraw-Hill Book Co., 1975.
7. Scott, N. R., *Electronic Computer Technology.* New York: McGraw-Hill Book Co., 1970, Chap. 10.
8. Kline, R. M., *Digital Computer Design.* Englewood Cliffs. N.J.: Prentice-Hall, Inc., 1977, Chap. 9.

PROBLEMS

7-1. The register of Fig. 7-1 transfers the input information into the flip-flops when the *CP* input goes through a positive-edge transition. Modify the circuit so that the input information is transferred into the register when a clock pulse goes through a negative-edge transition, provided a load input control is equal to binary 1.

7-2. The register of Fig. 7-3 loads the inputs during a negative transition of a clock pulse. What internal changes are necessary for the inputs to be loaded during the positive edge of a pulse?

7-3. Verify the circuit of Fig. 7-5 using maps to simplify the next-state equations.

7-4. Design a sequential circuit whose state diagram is given in Fig. 6-27 using a 3-bit register and a 16×4 ROM.

7-5. The content of a 4-bit shift register is initially 1101. The register is shifted six times to the right, with the serial input being 101101. What is the content of the register after each shift?

7-6. What is the difference between serial and parallel transfer? What type of register is used in each case?

7-7. The 4-bit bidirectional shift register of Fig. 7-9 is enclosed within one IC package.

(a) Draw a block diagram of the IC showing all inputs and outputs.

(b) Draw a block diagram using three ICs to produce a 12-bit bidirectional shift register.

7-8. The serial adder of Fig. 7-10 uses two 4-bit shift registers. Register *A* holds the binary number 0101 and register *B* holds 0111. The carry flip-flop *Q* is initially cleared. List the binary values in register *A* and flip-flop *Q* after each shift.

7-9. What changes are needed in the circuit of Fig. 7-11 to convert it to a circuit that subtracts the content of *B* from the content of *A*?

7-10. Design a serial counter; in other words, determine the circuit that must be included externally with a shift register in order to obtain a counter that operates in a serial fashion.

7-11. A flip-flop has a 20-ns delay from the time its *CP* input goes from 1 to 0 to the tune the output is complemented. What is the maximum delay in a 10-bit binary ripple counter that uses these flip-flops? What is the maximum frequency the counter can operate at reliably?

7-12. How many flip-flops must be complemented in a 10-bit binary ripple counter to reach the next count after 0111111111?

7-13. Draw the diagram of a 4-bit binary ripple down-counter using flip-flops that trigger on the (a) positive-edge transition and (b) negative-edge transition.

7-14. Draw a timing diagram similar to that in Fig. 7-15 for the binary ripple counter of Fig. 7-12.

7-15. Determine the next state for each of the six unused states in the BCD ripple counter of Fig. 7-14. Is the counter self-starting?

7-16. The ripple counter shown in Fig. P7-18 uses flip-flops that trigger on the negative-edge transition of the *CP* input. Determine the count sequence of the counter. Is the counter self-starting?

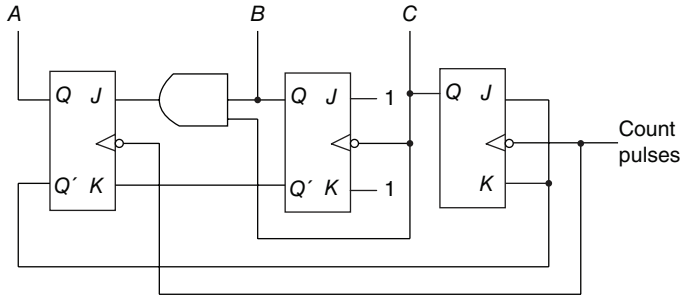


Figure P7-18 Ripple counter

7-17. What happens to the counter of Fig. 7-18 if both the up and down inputs are equal to 1 at the same time? Modify the circuit so that it will count up if this condition occurs.

7-18. Verify the flip-flop input functions of the synchronous BCD counter specified in Table 7-5. Draw the logic diagram of the BCD counter and include a count-enable control input.

7-19. Show the external connections of four IC binary counters with parallel load (Fig. 7-19) to produce a 16-bit binary counter. Use a block diagram for each IC.

7-20. Construct a BCD counter using the MSI circuit of Fig. 7-19.

7-21. Construct a mod-12 counter using the MSI circuit specified in Fig. 7-19. Give four alternatives.

7-22. Using two MSI circuits as specified in Fig. 7-19, construct a binary counter that counts from 0 to binary 64.

7-23. Using the *stop* variable from Fig. 7-21 as a start signal, construct a second word-time control that stays on for a period of 16 clock pulses.

7-24. Show that an n-bit binary counter connected to an n-to-2^n line decoder if equivalent to a ring counter with 2^n flip-flops. Draw the block diagrams of both circuits for $n = 3$. How many timing signals are generated?

7-25. Include an enable input to the decoder of Fig. 7-22(b) and connect it to the clock pulses. Draw the timing signals that are now generated at the outputs of the decoder.

7-26. Complete the design of the Johnson counter of Fig. 7-23, showing the outputs of the eight timing signals.

7-27. (a) List the eight unused states in the switch-tail ring counter of Fig. 7-23. Determine the next state for each unused state and show that, if the circuit finds itself in an invalid state, it does not return to a valid state, (b) Modify the circuit as recommended in the text and show that (1) the circuit produces the same sequence of states as listed in Fig. 7-23(b), and (2) the circuit reaches a valid state from any one of the unused states.

7-28. Construct a Johnson counter with ten timing signals.

7-29. (a) The memory unit of Fig. 7-24 has a capacity of 8192 words of 32 bits per word. How many flip-flops are needed for the memory address register and memory buffer register?

(b) How many words will the memory unit contain if the address register has 15 bits?

7-30. When the number of words to be selected in a memory is too large, it is convenient to use a binary storage cell with two select inputs; one X (horizontal) and one Y (vertical) select input. Both X and Y must be enabled to select the cell.

(a) Draw a binary cell similar to that in Fig. 7-29 with X and Y select inputs.

(b) Show how two 4×16 decoders can be used to select a word in a 256-word memory.

7-31. (a) Draw a block diagram of the 4×3 memory of Fig. 7-30, showing all inputs and outputs.

(b) Construct an 8×3 memory using two such units. Use a block diagram construction.

7-32. It is required to construct a memory with 256 words, 16 bits per word, organized as in Fig. 7-33. Cores are available in a matrix of 16 rows and 16 columns.

(a) How many matrices are needed?

(b) How many flip-flops are in the address and buffer registers?

(c) How many cores receive current during a read cycle?

(d) How many cores receive at least half-current during a write cycle?

7-33. Differentiate between Ripple counter and synchronous counter.

7-34. Write short notes on

(a) Registers

(b) Johnson counter

(c) Ring counter

(d) Random access memory

(e) Parallel in Serial Out (PISO) Shift registrar

7-35. Design a synchronous counter up-down with JK flip-flop which counts the odd number from 1 to 15 in binary.

7-36. Design a sequential circuit whose table is given bellow

Present Sate Input			Next State		Out put
A_1	A_2	x	A_1	A_2	y
0	0	0	0	1	0
0	0	1	1	1	0
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	1	1	1
1	0	1	0	1	0
1	1	0	1	0	0
1	1	1	0	0	0

7-37. Repeat the problem 4 with ROM and a register.

7-38. Design a MOD 10 synchronous binary up counter using T flip-flop and other necessary logic gates. Draw the timing diagram.

7-39. Design a 4-bit up/ down asynchronous counter using JK flip-flops and other necessary logic gates. Use one directional control input. If P = 0, the counter will count up and for P = 1, the counter will count down.

CHAPTER 8

Register-Transfer Logic

8.1 Introduction

A digital system is a sequential logic system constructed with flip-flops and gates. It was shown in previous chapters that a sequential circuit can be specified by means of a state table. To specify a large digital system with a state table would be very difficult, if not impossible, because the number of states would be prohibitively large. To overcome this difficulty, digital systems are invariably designed using a modular approach. The system is partitioned into modular subsystems, each of which performs some functional task. The modules are constructed from such digital functions as registers, counters, decoders, multiplexers, arithmetic elements, and control logic. The various modules are interconnected with common data and control paths to form a digital computer system. A typical digital system module would be the processor unit of a digital computer.

The interconnection of digital functions to form a digital system module cannot be described by means of combinational or sequential logic techniques. These techniques were developed to describe a digital system at the gate and flip-flop level and are not suitable for describing the system at the digital function level. To describe a digital system in terms of functions such as adders, decoders, and registers, it is necessary to employ a higher-level mathematical notation. The register-transfer logic method fulfills this requirement. In this method, the registers are selected to be the primitive components in the digital system, rather than the gates and flip-flops as in sequential logic. In this way it is possible to describe, in a concise and precise manner, the information flow and processing tasks among the data stored in the registers. The register-transfer logic method uses a set of expressions and statements which resemble the statements used in programming languages. This notation provides the necessary tools for specifying a prescribed set of interconnections between various digital functions. An important characteristic of the register-transfer logic method of presentation is that it is closely related to the way people would prefer to specify the operations of a digital system.

The basic components of this method are those that describe a digital system from the operational level. The operation of a digital system is best described by specifying:

1. The set of registers in the system and their functions.
2. The binary-coded information stored in the registers.
3. The operations performed on the information stored in the registers.
4. The control functions that initiate the sequence of operations.

These four components form the basis of the register-transfer logic method for describing digital systems.

A *register,* as defined in the register-transfer logic notation, not only implies a register as defined in Chapter 7, but also encompasses all other types of registers, such as shift registers, counters, and memory units. A counter is considered to be a register whose function is to increment by 1 the information stored within it. A memory unit is considered to be a collection of storage registers where information can be stored. A flip-flop standing alone is taken to be a 1-bit register. In fact, the flip-flops and associated gates of any sequential circuit are called a register by this method of designation.

The *binary information* stored in registers may be binary numbers, binary-coded decimal numbers, alphanumeric characters, control information, or any other binary-coded information. The operations that are performed on the data stored in registers depend on the type of data encountered. Numbers are manipulated with arithmetic operations, whereas control information is usually manipulated with logic operations such as setting and clearing specified bits in the register.

The operations performed on the data stored in registers are called *microoperations.* A microoperation is an elementary operation that can be performed in parallel during one clock pulse period. The result of the operation may replace the previous binary information of a register or may be transferred to another register. Examples of microoperations are shift, count, add, clear, and load. The digital functions introduced in Chapter 7 are registers that implement microoperations. A counter with parallel load is capable of performing the microoperations increment and load. A bidirectional shift register is capable of performing the shift-right and shift-left microoperations. The combinational MSI functions introduced in Chapter 5 can be used in some applications to perform microoperations. A binary parallel adder is useful for implementing the *add* microoperation on the contents of two registers that hold binary numbers. A microoperation requires only one clock pulse for execution if the operation is done in parallel. In serial computers, a microoperation requires a number of pulses equal to the word time in the system. This is equal to the number of bits in the shift registers that transfer the information serially while a microoperation is being executed.

The *control functions* that initiate the sequence of operations consist of timing signals that sequence the operations one at a time. Certain conditions which depend on results of previous operations may also determine the state of control functions. A control function is a binary variable that, when in one binary state, initiates an operation and, when in the other binary state, inhibits the operation.

The purpose of this chapter is to introduce the components of the register-transfer logic method in some detail. The chapter introduces a symbolic notation for representing registers, for specifying operations on the contents of registers, and for specifying control functions. This symbolic notation is sometimes called a *register-transfer language* or *computer hardware description language.* The register-transfer language adopted here is believed to be as simple as possible. It should be realized, however, that no standard symbology exists for a register-transfer language, and different sources adopt different conventions.

A statement in a register-transfer language consists of a control function and a list of microoperations. The control function (which may be omitted sometimes) specifies the control condition and timing sequence for executing the listed micro-operations. The microoperations specify the elementary operations to be performed on the information stored in registers. The

types of microoperations most often encountered in digital systems can be classified into four categories:

1. *Interregister-transfer* microoperations do not change the information content when the binary information moves from one register to another.
2. *Arithmetic* microoperations perform arithmetic on numbers stored in registers.
3. *Logic* microoperations perform operations such as AND and OR on individual pairs of bits stored in registers.
4. *Shift* microoperations specify operations for shift registers.

Sections 8-2 through 8-4 define a basic set of microoperations. Special symbols are assigned to the microoperations in the set, and each symbol is shown to be associated with corresponding digital hardware that implements the stated microoperation. It is important to realize that the register-transfer logic notation is directly related to, and cannot be separated from, the registers and the digital functions that it defines.

The microoperations performed on the information stored in registers depend on the type of data that reside in the registers. The binary information commonly found in registers of digital computers can be classified into three categories:

1. Numerical data such as binary numbers or binary-coded decimal numbers used in arithmetic computations.
2. Nonnumerical data such as alphanumeric characters or other binary-coded symbols used for special applications.
3. Instruction codes, addresses, and other control information used to specify the data-processing requirements in the system.

Sections 8-5 through 8-9 discuss the representation of numerical data and their relationship to the arithmetic microoperations. Section 8-10 explains the use of logic microoperations for processing nonnumerical data. The representation of instruction codes and their manipulation with microoperations are presented in Sections 8-11 and 8-12.

8.2 Interregister Transfer

The registers in a digital system are designated by capital letters (sometimes followed by numerals) to denote the function of the register. For example, the register that holds an address for the memory unit is usually called the memory address register and is designated *MAR*. Other designations for registers are *A*, *B*, *R*1, *R*2. and *IR.* The cells or flip-flops of an *n*-bit register are numbered in sequence from 1 to *n* (or from 0 to $n - 1$) starting either from the left or from the right. Figure 8-1 shows four ways to represent a register in block diagram form. The most common way to represent a register is by a rectangular box with the name of the register inside, as shown in Fig. 8-1(a). The individual cells can be distinguished as in (b), with each cell assigned a letter with a subscript number. The numbering of cells from right to left can be marked on top of the box as in the 12-bit register *MBR* in (c). A 16-bit register is partitioned into two parts in (d). Bits 1 through 8 are assigned the symbol letter *L* (for low) and bits 9 through 16 are assigned the symbol letter *H* (for high). The name of the 16-bit register is *PC.* The symbol *PC*(*H*) refers to the eight high-ordered cells and *PC*(*L*) refers to the eight low-ordered cells of the register.

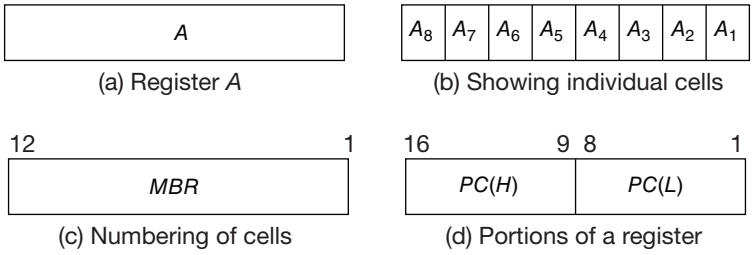


Figure 8.1 Block diagram of registers

Registers can be specified in a register-transfer language with a declaration statement. For example, the registers of Fig. 8-1 can be defined with declaration statements such as:

DECLARE REGISTER $A(8)$, $MBR(12)$, $PC(16)$
DECLARE SUBREGISTER $PC(L) = PC(1{-}8)$, $PC(H) = PC(9{-}16)$

However, in this book we will not use declaration statements to define registers; instead, the registers will be shown in block diagram form as in Fig. 8-1. Registers shown in a block diagram can be easily converted into declaration statements for simulation purposes.

Information transfer from one register to another is designated in symbolic form by means of the *replacement operator.* The statement:

$$A \leftarrow B$$

denotes the transfer of the *contents* of register *B* into register *A.* It designates a replacement of the contents of *A* by the contents of *B.* By definition, the contents of the source register *B* do not change after the transfer.

A statement that specifies a register transfer implies that circuits are available from the outputs of the source register to the cell inputs of the destination register. Normally, we do not want this transfer to occur with every clock pulse, but only under a predetermined condition. The condition that determines when the transfer is to occur is called a *control junction.* A control function is a Boolean function that can be equal to 1 or 0. The control function is included with the statement as follows:

$$x'T_1: A \leftarrow B$$

The control function is terminated with a colon. It symbolizes the requirement that the transfer operation be executed by the hardware only when the Boolean function $x'T_1 = 1$, i.e., when variable $x = 0$ and timing variable $T_1 = 1$.

Every statement written in a register-transfer language implies a hardware construction for implementing the transfer. Figure 8-2 shows the implementation of the statement written above. The outputs of register *B* are connected to the inputs of register *A*, and the number of lines in this connection is equal to the number of bits in the registers. Register *A* must have a load control input so that it can be enabled when the control function is 1. Although not shown, it is assumed that register *A* has an additional input that accepts continuous synchronized clock pulses. The control function is generated by means of an inverter and an AND gate. It is also assumed that the

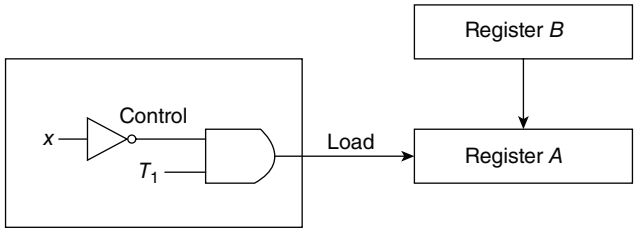


Figure 8.2 Hardware implementation of the statement $x'T_1$: $A \leftarrow B$

control unit that generates the timing variable T_1, is synchronized with the same clock pulses that are applied to register *A*. The control function stays on during one clock pulse period (when the timing variable is equal to 1), and the transfer occurs during the next transition of a clock pulse.

The basic symbols of the register-transfer logic are listed in Table 8-1. Registers are denoted by capital letters, and numerals may follow the letters. Subscripts are used to distinguish individual cells of the register. Parentheses are used to define a portion of a register. The arrow denotes a transfer of information and the direction of transfer. A colon terminates a control function, and the comma is used to separate two or more operations that are executed at the same time. The statement:

$$T_3\colon A \leftarrow B, \qquad B \leftarrow A$$

denotes an exchange operation that swaps the contents of two registers during one common clock pulse. This simultaneous operation is possible in registers with master-slave or edge-triggered flip-flops.

The square brackets are used in conjunction with memory transfer. The letter *M* designates a memory word, and the register enclosed inside the square brackets provides the address for the memory. This is explained in more detail below.

There are occasions when a destination register receives information from two sources, but evidently not at the same time. Consider the two statements:

$$T_1\colon C \leftarrow A$$
$$T_5\colon C \leftarrow B$$

Table 8-1 Basic symbol for register-transfer logic

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	*A, MBR, R*2
Subscript	Denotes a bit of a register	A_2, B_6
Parentheses ( )	Denotes a portion of a register	*PC*(*H*), *MBR*(*OP*)
Arrow ←	Denotes transfer of information	$A \leftarrow B$
Colon :	Terminates a control function	$x'T_0$:
Comma ,	Separates two microoperations	$A \leftarrow B, B \leftarrow A$
Square brackets [ ]	Specifies an address for memory transfer	$MBR \leftarrow M[MAR]$

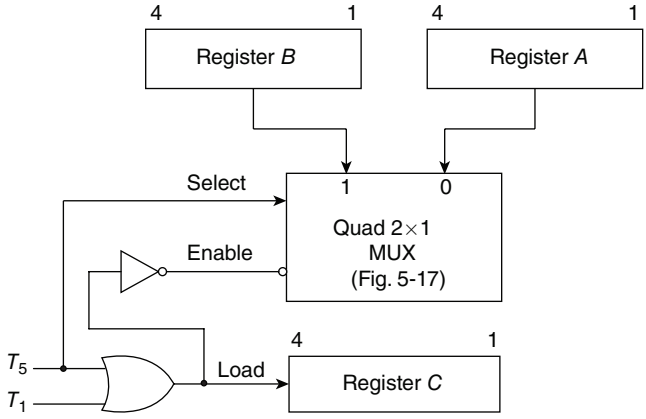


Figure 8.3 Use of a multiplexer to transfer information from two sources into a single destination

The first line states that the contents of register A are to be transferred to register C when timing variable T_1 occurs. The second statement uses the same destination register as the first, but with a different source register and a different timing variable. The connection of two source registers to the same destination register cannot be done directly, but requires a multiplexer circuit to select between two possible paths. The block diagram of the circuit that implements the two statements is shown in Fig. 8-3. For registers with four bits each, we need a quadruple 2-to-l line multiplexer, similar to the one previously given in Fig. 5-17, in order to select either register A or register B. When $T_5 = 1$, register B is selected, but when $T_1 = 1$, register A is selected (because T_5 must be 0 when T_1 is 1). The multiplexer and the load input of register C are enabled every time T_1 or T_5 occurs. This causes a transfer of information from the selected source register into the destination register.

8.2.1 Bus Transfer

Quite often a digital system has many registers, and paths must be provided to transfer information from one register to another register. Consider, for example, the requirement for transfer among three registers as shown in Fig. 8-4. There are six data paths, and each register requires a multiplexer to select between two sources. If each register consists of n flip-flops, there is a need for $6n$ lines and three multiplexers. As the number of registers increases, the number of interconnection lines and multiplexers increases. If we restrict the transfer to one at a time, the number of paths among the registers can be reduced considerably. This is shown in Fig. 8-5, where the

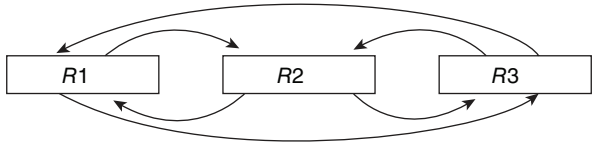


Figure 8.4 Transfer among three registers

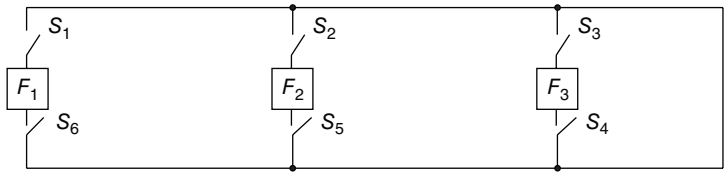


Figure 8.5 Transfer through one common line

output and input of each flip-flop is connected to a common line through an electronic circuit that acts like a switch. All the switches are normally open until a transfer is required. For a transfer from F_1 to F_3, for example, switches S_1 and S_4 are closed to form the required path. This scheme can be extended to registers with n flip-flops, and it requires n common lines.

A group of wires through which binary information is transferred one at a time among registers is called a *bus.* For parallel transfer, the number of lines in the bus is equal to the number of bits in the registers. The idea of a bus transfer is analogous to a central transportation system used to bring commuters from one point to another. Instead of each commuter using private transportation to go from one location to another, a bus system is used, and the commuters wait in line until transportation is available.

A common-bus system can be constructed with multiplexers, and a destination register for the bus transfer can be selected by means of a decoder. The multiplexers select one source register for the bus, and the decoder selects one destination register to transfer the information from the bus. The construction of a bus system for four registers is depicted in Fig. 8-6. The four bits in the same significant position in the registers go through a 4-to-l line multiplexer to form one line of the bus. Only two multiplexers are shown in the diagram: one for the low-order significant bits and one for the high-order significant bits. For registers of n bits, n multiplexers are needed to produce an n-line bus. The n lines in the bus are connected to the n inputs of ail registers. The transfer of information from the bus into one destination register is accomplished by activating the load control of that register. The particular load control activated is selected by the outputs of the decoder when enabled. If the decoder is not enabled, no information will be transferred, even though the multiplexers place the contents of a source register onto the bus.

To illustrate with a particular example, consider the statement:

$$C \leftarrow A$$

The control function that enables this transfer must select register A for the bus and register C for the destination. The multiplexers and decoder select inputs must be:

Select source = 00	(MUXs select register A)
Select destination = 10	(decoder selects register C)
Decoder enable = 0	(decoder is enabled)

On the next clock pulse, the contents of A, being on the bus, are loaded into register C.

8.2.2 Memory Transfer

The operation of a memory unit was described in Section 7-7. The transfer of information from a memory register to the outside environment is called a *read* operation. The transfer of new

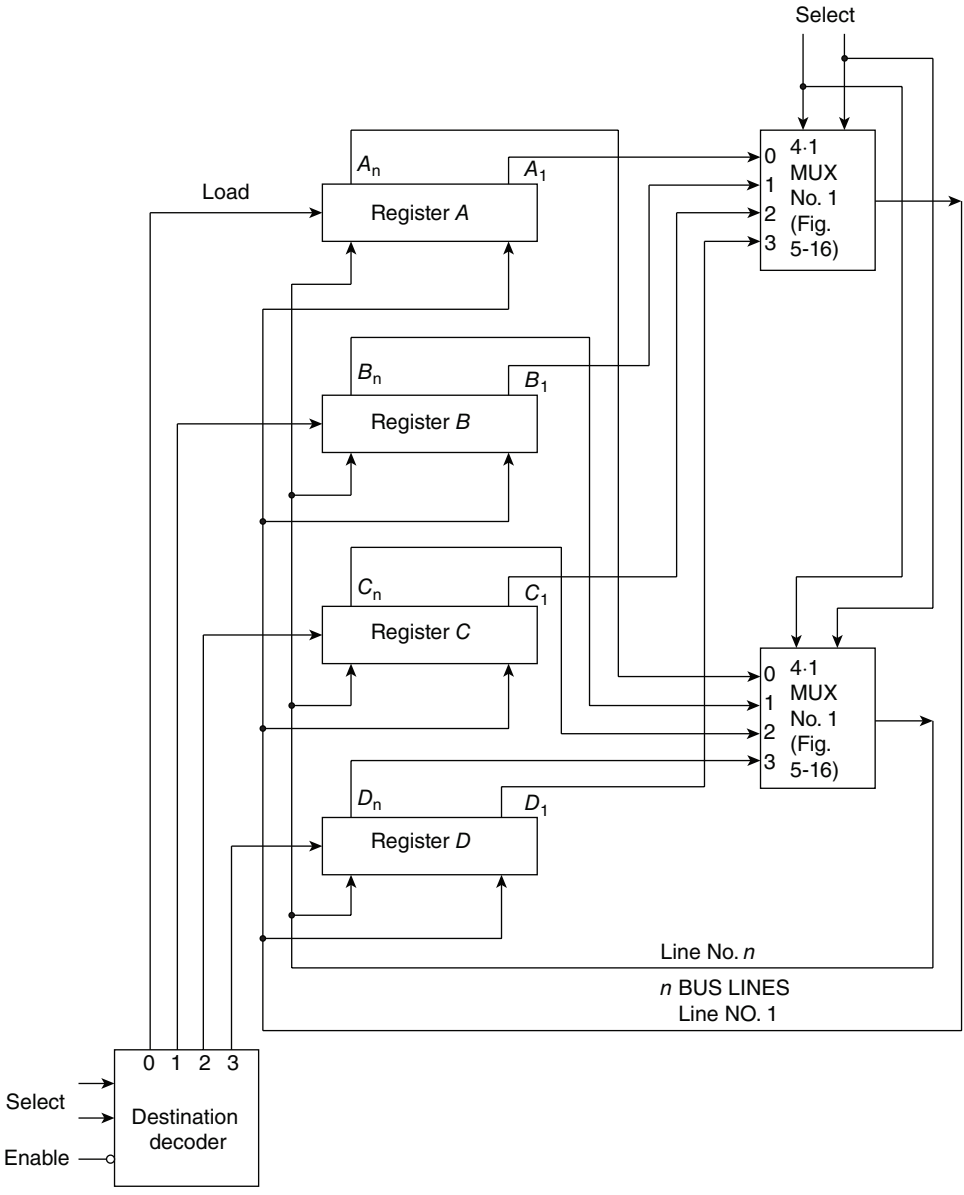


Figure 8.6 Bus system for four registers

information into a memory register is called a *write* operation. In both operations, the memory register selected is specified by an address.

A memory register or word is symbolized by the letter *M.* The particular memory register among the many available in a memory unit is selected by the memory address during the transfer. It is necessary to specify the address of *M* when writing memory-transfer statements. In

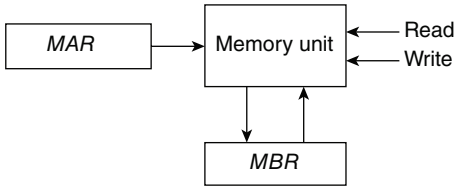


Figure 8.7 Memory unit that communicates with two external registers

some applications, only one address register is connected to the address terminals of the memory. In other applications, the address lines form a common-bus system to allow many registers to specify an address. When only one register is connected to the memory address, we know that this register specifies the address and we can adopt a convention that will simplify the notation. If the letter *M* stands by itself in a statement, it will always designate a memory register selected by the address presently in *MAR*. Otherwise, the register that specifies the address (or the address itself) will be enclosed within square brackets after the symbol *M*.

Consider a memory unit that has a single address register, *MAR*, as shown in Fig. 8-7. The diagram also shows a single memory buffer register *MBR* used to transfer data into and out of memory. There are two memory-transfer operations: read and write. The read operation is a transfer from the selected memory register *M* into *MBR*. This is designated symbolically by the statement:

$$R: MBR \leftarrow M$$

R is the control function that initiates the read operation. This causes a transfer of information into *MBR* from the selected memory register *M* specified by the address in *MAR*. The write operation is a transfer from *MBR* to the selected memory register *M*. This is designated by the statement:

$$W: M \leftarrow MBR$$

W is the control function that initiates the write operation. This causes a transfer of information from *MBR* into the memory register *M* selected by the address presently in *MAR*.

The access time of a memory unit must be synchronized with the master clock pulses in the system that triggers the processor registers. In fast memories, the access time may be shorter than or equal to a clock pulse period. In slow memories, it may be necessary to wait for a number of clock pulses for the transfer to be completed. In magnetic-core memories, the processor registers must wait for the memory cycle time to be completed. For a read operation, the cycle time includes the restoration of the word after reading. For a write operation, the cycle time includes the clearing of the memory word prior to writing.

In some systems, the memory unit receives addresses and data from many registers connected to common buses. Consider the case depicted in Fig. 8-8. The address to the memory unit comes from an address bus. Four registers are connected to this bus and any one may supply an address. The output of the memory can go to any one of four registers which are selected by a decoder. The data input to the memory comes from the data bus, which selects one of four registers. A memory word is specified in such a system by the symbol *M* followed by a register enclosed in

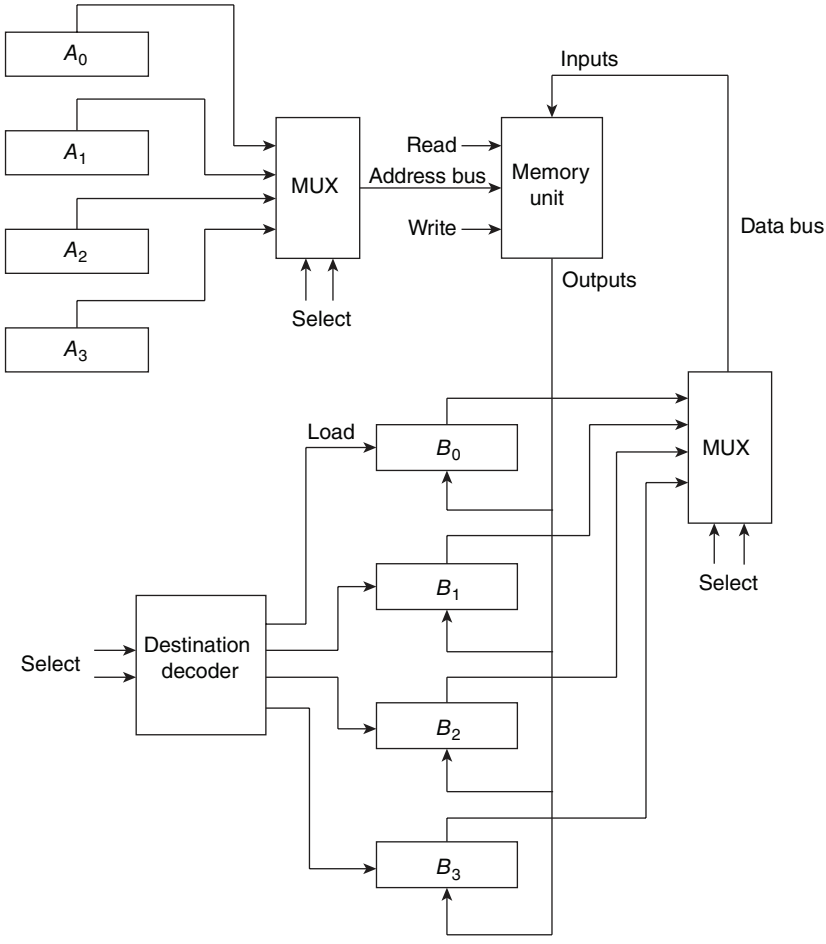


Figure 8.8 Memory unit that communicates with multiple registers

square brackets. The contents of the register within the square brackets specify the address for *M*. The transfer of information from register *B*2 to a memory word selected by the address in register *A*1 is symbolized by the statement:

$$W\text{:}\ M[A1] \leftarrow B2$$

This is a write operation, with register *A*1 specifying the address. The square brackets after the letter *M* give the address register used for selecting the memory register *M*. The statement does not specify the buses explicitly. Nevertheless, it implies the required selection inputs for the two multiplexers that form the address and data buses.

The read operation in a memory with buses can be specified in a similar manner. The statement:

$$R\text{:}\ B0 \leftarrow M[A3]$$

symbolizes a read operation from a memory register whose address is given by $A3$. The binary information coming out of memory is transferred to register $B0$. Again, this statement implies the required selection inputs for the address multiplexer and the selection variables for the destination decoder.

8.3 Arithmetic, Logic, and Shift Microoperations

The interregister-transfer microoperations do not change the information content when the binary information moves from the source register to the destination register. All other microoperations change the information content during the transfer. Among all possible operations that can exist in a digital system, there is a basic set from which all other operations can be obtained. In this section, we define a set of basic microoperations, their symbolic notation, and the digital hardware that implements them. Other microoperations with appropriate symbols can be defined if necessary to suit a particular application.

8.3.1 Arithmetic Microoperations

The basic arithmetic microoperations are add, subtract, complement, and shift. Arithmetic shifts are explained in Section 8-7 in conjunction with the type of binary data representation. All other arithmetic operations can be obtained from a variation or a sequence of these basic microoperations.

The arithmetic microoperation defined by the statement:

$$F \leftarrow A + B$$

specifies an *add* operation. It states that the contents of register A are to be added to the contents of register B, and the sum transferred to register F. To implement this statement, we require three registers, A, B, and F, and the digital function that performs the addition operation, such as a parallel adder. The other basic arithmetic operations are listed in Table 8-2. Arithmetic subtraction implies the availability of a binary parallel subtractor composed of full-subtractor circuits connected in cascade. Subtraction is most often implemented through complementation and addition as specified by the statement:

$$F \leftarrow A + \overline{B} + 1$$

Table 8-2 Arithmetic microoperations

Symbolic designation	Description
$F \leftarrow A + B$	Contents of A plus B transferred to F
$F \leftarrow A - B$	Contents of A minus B transferred to F
$B \leftarrow \overline{B}$	Complement register B (1's complement)
$B \leftarrow \overline{B} + 1$	Form the 2's complement of the contents of register B
$F \leftarrow A + \overline{B} + 1$	A plus the 2's complement of B transferred to F
$A \leftarrow A + 1$	Increment the contents of A by 1 (count up)
$A \leftarrow A - 1$	Decrement the contents of A by 1 (count down)

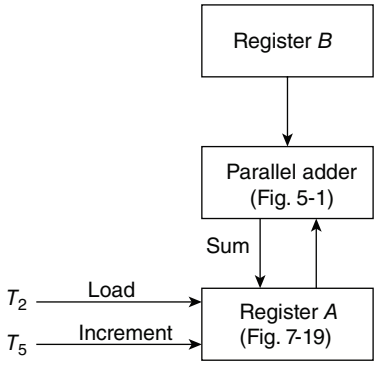


Figure 8.9 Implementation for the add and increment microoperations

$\overline{B}$ is the symbol for the 1's complement of *B*. Adding 1 to the 1's complement gives the 2's complement of *B*. Adding *A* to the 2's complement of *B* produces *A* minus *B*.

The increment and decrement microoperations are symbolized by a *plus-one* or *minus-one* operation executed on the contents of a register. These microoperations are implemented with an up-counter or down-counter, respectively.

There must be a direct relationship between the statements written in a register-transfer language and the registers and digital functions which are required for their implementation. To illustrate this relationship, consider the two statements:

$$T_2: A \leftarrow A + B$$
$$T_5 A \leftarrow A + 1$$

Timing variable T_2 initiates an operation to add the contents of register *B* to the present contents of *A*. Timing variable T_5 increments register *A*. The incrementing can be easily done with a counter, and the sum of two binary numbers can be generated with a parallel adder. The transfer of the sum from the parallel adder into register *A* can be activated with a load input in the register. This dictates that the register be a counter with parallel-load capability. The implementation of the above two statements is shown in block diagram form in Fig. 8-9. A parallel adder receives input information from registers *A* and *B*. The sum bits from the parallel adder are applied to the inputs of *A*, and timing variable T_2 loads the sum into register *A*. Timing variable T_5 increments the register by enabling the increment input (or count input, as in Fig. 7-19).

Note that the arithmetic operations multiply and divide are not listed in Table 8-2. The multiplication operation can be represented by the symbol *, and the division by a /. These two operations are valid arithmetic operations but are not included in the basic set of microoperations. The only place where these operations can be considered as microoperations is in a digital system where they are implemented by means of combinational circuits. In such a case, the signals that perform these operations propagate through gates, and the result of the operation can be transferred into a destination register by a clock pulse as soon as the output signals progagate through the combinational circuit. In most computers, the multiplication operation is implemented with a sequence of add and shift microoperations. Division is implemented with a sequence of subtract and shift microoperations. To specify the hardware implementation in such a case requires a list of statements that use the basic microoperations of *add, subtract,* and *shift*.

8.3.2 Logic Microoperations

Logic microoperations specify binary operations for a string of bits stored in registers. These operations consider each bit in the registers separately and treat it as a binary variable. As an illustration, the exclusive-OR microoperation is symbolized by the statement:

$$F \leftarrow A \oplus B$$

It specifies a logic operation that considers each pair of bits in the registers as binary variables. If the content of register A is 1010 and that of register B 1100, the information transferred to register F is 0110:

1010	content of A
1100	content of B
0110	content of $F \leftarrow A \oplus B$

There are 16 different possible logic operations that can be performed with two binary variables. These logic operations are listed in Table 2-6. All 16 logic operations can be expressed ill terms of the AND, OR, and complement operations. Special symbols will be adopted for these three microoperations to distinguish them from the corresponding symbols used to express Boolean functions. The symbol $\vee$ will be used to denote an OR microoperation and the symbol $\wedge$ to denote an AND microoperation. The complement microoperation is the same as the 1's complement and uses a bar on top of the letter (or letters) that denotes the register. By using these symbols, it will be possible to differentiate between a logic microoperation and a control (or Boolean) function. The symbols for the four logic microoperations are summarized in Table 8-3. The last two symbols are for the shift microoperations discussed below.

A more important reason for adopting a special symbol for the OR microoperation is to differentiate the symbol +, when used as an arithmetic plus, from a logic OR operation. Although the + symbol has two meanings, it will be possible to distinguish between them by noting where the symbol occurs. When this symbol occurs in a microoperation, it denotes an arithmetic plus. When it occurs in a control (or Boolean) function, it denotes a logic OR operation. For example, in the statement:

$$T_1 + T_2: A \leftarrow A + B,\ C \leftarrow D \vee F$$

Table 8-3 Logic and shift microoperations

Symbolic designation	Description
$A \leftarrow \overline{A}$	Complement all bits of register A
$F \leftarrow A \vee B$	Logic OR microoperation
$F \leftarrow A \wedge B$	Logic AND microoperation
$F \leftarrow A \oplus B$	Logic exclusive-OR microoperation
$A \leftarrow \text{shl } A$	Shift-left register A
$A \leftarrow \text{shr } A$	Shift-right register A

the + between T_1 and T_2 is an OR operation between two timing variables of a control function. The + between A and B specifies an add microoperation. The OR microoperation is designated by the symbol $\vee$ between registers D and F.

The logic microoperations can be easily implemented with a group of gates. The complement of a register of n bits is obtained from n inverter gates. The AND microoperation is obtained from a group of AND gates, each of which receives a pair of bits from the two source registers. The outputs of the AND gates are applied to the inputs of the destination register. The OR microoperation requires a group of OR gates arranged in a similar fashion.

8.3.3 Shift Microoperations

Shift microoperations transfer binary information between registers in serial computers. They are also used in parallel computers for arithmetic, logic, and control operations. Registers can be shifted to the left or to the right. There are no conventional symbols for the shift operations. In this book, we adopt the symbols *shl* and *shr* for the shift-left and shift-right operations, respectively, For example:

$$A \leftarrow \text{shl } A, \qquad B \leftarrow \text{shr } B$$

are two microoperations that specify a l-bit shift to the left of register A and a 1-bit shift to the right of register B. The register symbol must be the same on both sides of the arrow as in the increment operation.

While the bits of a register are shifted, the extreme flip-flops receive information from the serial input. The extreme flip-flop is in the leftmost position of the register during a shift-right operation and in the rightmost position during a shift-left operation. The information transferred into the extreme flip-flops is not specified by the *shl* and *shr* symbols. Therefore, a shift microoperation statement must be accompanied with another microoperation that specifies the value of the serial input for the bit transfer into the extreme flip-flop. For example:

$$A \leftarrow \text{shl } A, \; A_1 \leftarrow A_n$$

is a circular shift chat transfers the leftmost bit from A_n into the rightmost flip-flop A_1 Similarly:

$$A \leftarrow \text{shr } A, \; A_n \leftarrow E$$

is a shift-right operation with the leftmost flip-flop A_n receiving the value of the 1-bit register E.

8.4 Conditional Control Statements

It is sometimes convenient to specify a control condition by a conditional statement rather than a Boolean control function. A *conditional control* statement is symbolized by an *if-then-else* statement in the following manner:

P: If (condition) *then* [microoperation(s)] *else* [microoperation(s)]

The statement is interpreted to mean that if the control condition stated within the parentheses after the word *if* is true, then the microoperation (or microoperations) enclosed within the parentheses after the word *then* is executed. If the condition is not true, the microoperation listed after

the word *else* is executed. In any case, the control function P must occur for anything to be done. If the *else* part of the statement is missing, then nothing is executed if the condition is not true.

The conditional control statement is more of a convenience than a necessity. It enables the writing of clearer statements that are easier for people to interpret. It can always be rewritten in a conventional statement without an if-then-else form. As an example, consider the conditional control statement:

$$T_2\text{: If } (C = 0) \text{ then } (F \leftarrow 1) \text{ else } (F \leftarrow 0)$$

F is assumed to be a 1-bit register (flip-flop) that can be set or cleared. If register C is a 1-bit register, the statement is equivalent to the following two statements:

$$C'T_2\text{: } F \leftarrow 1$$
$$CT_2\text{: } F \leftarrow 0$$

Note that the same timing variable can occur in two separate control functions. The variable C can be either 0 or 1; therefore, only one of the microoperations will be executed during T_2, depending on the value of C.

If register C has more than one bit, the condition $C = 0$ means that all bits of C must be 0. Assume that register C has four bits C_1, C_2, C_3, and C_4. The condition for $C = 0$ can be expressed with a Boolean function:

$$x = C'_1C'_2C'_3C'_4 = (C_1 + C_2 + C_3 + C_4)'$$

Variable x can be generated with a NOR gate. Using the definition of x as above, the conditional control statement is now equivalent to the two statements;

$$xT_2\text{: } \quad F \leftarrow 1$$
$$x'T_2\text{: } \quad F \leftarrow 0$$

Variable $x = 1$ if $C = 0$ but is equal to 0 if $C \neq 0$.

When writing conditional control statements, one must realize that the condition stated after the word *if* is part of the control function and not part of a microoperation statement. The condition must be clearly stated and must be implementable with a combinational circuit,

8.5 Fixed-point Binary Data

The binary information found in registers represents either data or control information. Data are operands and other discrete elements of information operated on to achieve required results. Control information is a bit or group of bits that specify the operations to be done. A unit of control information stored in digital computer registers is called an *instruction* and is a binary code that specifies the operations to be performed on the stored data. Instruction codes and their representation in registers are presented in Section 8-11. Some commonly used types of data and their representation in registers are presented in this and the following sections.

8.5.1 Sign and Radix-Point Representation

A register with n flip-flops can store a binary number of n bits; each flip-flop represents one binary digit. This represents the magnitude of the number but does not give information about its

sign or the position of the binary point. The sign is needed for arithmetic operations, as it shows whether the number is positive or negative. The position of the binary point is needed to represent integers, fractions, or mixed integer-fraction numbers.

The sign of a number is a discrete quantity of information having two values: plus and minus. These two values can be represented by a code of one bit. The convention is to represent a plus with a 0 and a minus with a 1. To represent a sign binary number in a register, we need $n = k + 1$ flip-flops, k flip-flops for the magnitude and one for storing the sign of the number.

The representation of the binary point is complicated by the fact that it is characterized by a *position* between two flip-flops in the register. There are two possible ways of specifying the position of the binary point in a register: by giving it a *fixed-point* position or by employing a *floating-point* representation. The fixed-point method assumes that the binary point is always fixed in one position. The two positions most widely used are (1) a binary point in the extreme left of the register to make the stored number a fraction, and (2) a binary point in the extreme right of the register to make the stored number an integer. In both cases, the binary point is not physically visible but is assumed from the fact that the number stored in the register is treated as a fraction or as an integer. The floating-point representation uses a second register to store a number that designates the position of the binary point in the first register. Floating-point representation is explained in Section 8-9.

8.5.2 Signed Binary Numbers

When a fixed-point binary number is positive, the sign is represented by 0 and the magnitude by a positive binary number. When the number is negative, the sign is represented by 1 and the rest of the number may be represented in any one of three different ways. These are:

1. Sign-magnitude.
2. Sign-1's complement.
3. Sign-2's complement.

In the sign-magnitude representation, the magnitude is represented by a positive binary number. In the other two representations, the number is in either 1's or 2's complement. If the number is positive, the three representations are the same.

As an example, the binary number 9 is written below in the three representations. It is assumed that a 7-bit register is available to store the sign and the magnitude of the number.

	+ 9	− 9
Sign-magnitude	0 001001	1 001001
Sign-1's complement	0 001001	1 110110
Sign-2's complement	0 001001	1 110111

A positive number in any representation has a 0 in the leftmost bit for a plus, followed by a positive binary number. A negative number always has a 1 in the leftmost bit for a minus, but the magnitude bits are represented differently. In the sign-magnitude representation, these bits are the positive number; in the 1's-complement representation, these bits are the complement of the binary number; and in the 2's-complement representation, the number is in its 2's-complement form.

The sign-magnitude representation of − 9 is obtained from +9 (0 001001) by complementing *only* the sign bit. The sign-1's-complement representation of − 9 is obtained by complementing *all* the bits of 0 001001 (+ 9), including the sign bit. The sign-2's-complement representation is obtained by taking the 2's complement of the positive number, *including* its sign bit.

8.5.3 Arithmetic Addition

The reason for using the sign-complement representation for negative numbers will become apparent after we consider the steps involved in forming the sum of two signed numbers. The sign-magnitude representation is the one used in everyday calculations. For example, +23 and −35 are represented with a sign, followed by the magnitude of the number. To add these two numbers, it is necessary to subtract the smaller magnitude from the larger magnitude and to use the sign of the larger number for the sign of the result, i.e., (+ 23) + (− 35) = − (35 − 23) = − 12. The process of adding two signed numbers when negative numbers are represented in sign-magnitude form requires that we compare their signs. If the two signs are the same, we add the two magnitudes. If the signs are not the same, we compare the relative magnitudes of the numbers and then subtract the smaller from the larger. It is necessary also to determine the sign of the result. This is a process that, when implemented with digital hardware, requires a sequence of control decisions as well as circuits that can compare, add, and subtract numbers.

Now compare the above procedure with the procedure that forms the sum of two signed binary numbers when negative numbers are in 1's- or 2's-complement representation. These procedures are very simple and can be stated as follows:

Addition with sign-2's-complement representation. The addition of two signed binary numbers with negative numbers represented by their 2's complement is obtained from the addition of the two numbers including their sign bits. A carry in the most significant (sign) bit is discarded.

Addition with sign-1's-complement representation. The addition of two signed binary numbers with negative numbers represented by their 1's complement is obtained from the addition of the two numbers, including their sign bits. If there is a carry out of the most significant (sign) bit, the result is incremented by 1 and its carry is discarded.

Numerical examples for addition with negative numbers represented by their 2's complement are shown below. Note that negative numbers must be initially in 2's-complement representation and that the sum obtained after addition is always in the required representation.

+ 6	0 000110		− 6	1 111010	
		+			+
+ 9	0 001001		+ 9	0 001001	
+ 15	0 001111		+ 3	0 000010	
+ 6	0 000110	+	− 9	1 110111	
					+
− 9	1 110111		− 9	1 110111	
− 3	1 111101		− 18	1 101110	

The two numbers in the four examples are added, including their sign bits. Any carry out of the sign bit is discarded, and negative results are automatically in their 2's-complement form.

The four examples are repeated below with negative numbers represented by their 1's complement. The carry out of the sign bit is returned and added to the least significant bit (end-around carry).

+ 6	0 000110		− 6	1 111001	
		+			+
+ 9	0 001001		+ 9	0 001001	
+ 15	0 001111			10 000010	
					+
				→ 1	
			+ 3	0 000011	
+ 6	0 000110	+	− 9	1 110110	
					+
− 9	1 110110		− 9	1 110110	
− 3	1 111100			11 101100	
					+
				→ 1	
			− 18	1 101101	

The advantage of the sign-2's-complement representation over the sign-1's-complement form (and the sign-magnitude form) is that it contains only one type of zero. The other two representations have both a positive zero and a negative zero. For example, adding + 9 to − 9 in the 1's-complement representation, one obtains:

+ 9	0 001001
− 9	1 110110
− 0	1 111111

and the result is a negative zero, i.e., the complement of 0 000000 (positive zero). A zero with an associated sign bit will appear in a register in one of the following forms, depending on the representation used for negative numbers:

	+ 0	− 0
In sign-magnitude	0 0000000	1 0000000
In sign-1's complement	0 0000000	1 1111111
In sign-2's complement	0 0000000	none

Both the sign-magnitude and the 1's-complement representations have associated with them the possibility of a negative zero. The sign-2's-complement representation has only a positive zero. This occurs because the 2's complement of 0 000000 (positive zero) is 0 000000 and

may be obtained from the 1's complement plus 1 (i.e., 1 111111 + 1) provided the end carry is discarded.

The range of binary integer numbers that can be accommodated in a register of $n = k + 1$ bits is $\pm (2^k - 1)$, where k bits are reserved for the number and one bit for the sign. A register with 8 bits can store binary numbers in the range of $\pm (2^7 - 1) = \pm 127$. However, since the sign-2's-complement representation has only one zero, it should accommodate one more number than the other two representations. Consider the representation of the largest and smallest numbers:

	sign-1's complement	sign-2's complement
+ 126 = 0 1111110	− 126 = 1 0000001	1 0000010
+ 127 = 0 1111111	− 127 = 1 0000000	1 0000001
+ 128 (impossible)	− 128 (impossible)	1 0000000

In the sign-2's-complement representation, it is possible to represent −128 with eight bits. In general, the sign-2's-complement representation can accommodate numbers in the range $+ (2^k - 1)$ to -2^k, where $k = n - 1$ and n is the number of bits in the register.

8.5.4 Arithmetic Subtraction

Subtraction of two signed binary numbers when negative numbers are in the 2's-complement form is very simple and can be stated as follows: *Take the* 2's *complement of the subtrahend* (*including the sign bit*) *and add it to the minuend* (*including sign bit*). This procedure uses the fact that a subtraction operation can be changed to an addition operation if the sign of the subtrahend is changed. This is demonstrated by the following relationships (B is the subtrahend):

$$(\pm A) - (-B) = (\pm A) + (+B)$$
$$(\pm A) - (+B) = (\pm A) + (-B)$$

But changing a positive number to a negative number is easily done by taking its 2's complement (including the sign bit). The reverse is also true because the complement of the complement restores the number to its original value.

The subtraction with l's-complement numbers is similar except for the end-around carry. Subtraction with sign-magnitude requires that only the sign bit of the subtrahend be complemented. Addition and subtraction of binary numbers in sign-magnitude representation is demonstrated in Section 10-3.

Because of the simple procedure for adding and subtracting binary numbers when negative numbers are in sign-2's-complement form, most computers adopt this representation over the more familiar sign-magnitude form. The reason 2's complement is usually chosen over the 1's complement is to avoid the end-around carry and the occurrence of a negative zero.

8.6 Overflow

When two numbers of n digits each are added and the sum occupies $n + 1$digits, we say that an *overflow* occurs. This is true for binary numbers or decimal numbers whether signed or unsigned. When one performs the addition with paper and pencil, an overflow is not a problem, since we

are not limited by the width of the page to write down the sum. An overflow is a problem in a digital computer because the lengths of all registers, including memory registers, are of finite length. A result of $n + 1$ bits cannot be accommodated in a register of standard length n. For this reason, many computers check for the occurrence of an overflow, and when it occurs, they set an overflow flip-flop for the user to check.

An overflow cannot occur after an addition if one number is positive and the other is negative, since adding a positive number to a negative number produces a result (positive or negative) which is smaller than the larger of the two original numbers. An overflow may occur if the two numbers are added and both are positive or both are negative. When two numbers in sign-magnitude representation are added, an overflow can be easily detected from the carry out of the number bits. When two numbers in sign-2's-complement representation are added, the sign bit is treated as part of the number and the end carry does not necessarily indicate an overflow.

The algorithm for adding two numbers in sign-2's-complement representation, as previously stated, gives an incorrect result when an overflow occurs. This arises because an overflow of the number bits always changes the sign of the result and gives an erroneous n-bit answer. To see how this happens, consider the following example. Two signed binary numbers, 35 and 40, are stored in two 7-bit registers. The maximum capacity of the register is $(2^6 - 1) = 63$ and the minimum capacity is $-2^6 = -64$. Since the sum of the numbers is 75, it exceeds the capacity of the register. This is true if the numbers are both positive or both negative. The operations in binary are shown below together with the last two carries of the addition:

	carries: 0 1		carries: 1 0
+ 35	0 100011	– 35	1 011101
+ 40	0 101000	– 40	1 011000
+ 75	1 001011	– 75	0 110101

In either case, we see that the 7-bit result that should have been positive is negative, and vice versa. Obviously, the binary answer is incorrect and the algorithm for adding binary numbers represented in 2's complement as stated previously fails to give correct results when an overflow occurs. Note that if the carry out of the sign-bit position is taken as the sign for the result, then the 8-bit answer so obtained will be correct.

An overflow condition can be detected by observing the carry *into* the sign-bit position and the carry *out* of the sign-bit position. If these two carries are not equal, an overflow condition is produced. This is indicated in the example above where the two carries are explicitly shown. The reader can try a few examples of numbers that do not produce an overflow to see that these two carries will always come out to be either both 0's or both 1's. If the two carries are applied to an exclusive-OR gate, an overflow would be detected when the output of the gate is 1.

The addition of two signed binary numbers when negative numbers are represented in sign-2's-complement form is implemented with digital functions as shown in Fig. 8-10. Register A holds the augend, with the sign bit in position A_n. Register B holds the addend, with the sign bit in B_n. The two numbers are added by means of an n-bit parallel adder. The full-adder (FA) circuit in stage n (the sign bits) is shown explicitly. The carry going into this full-adder is C_n. The carry

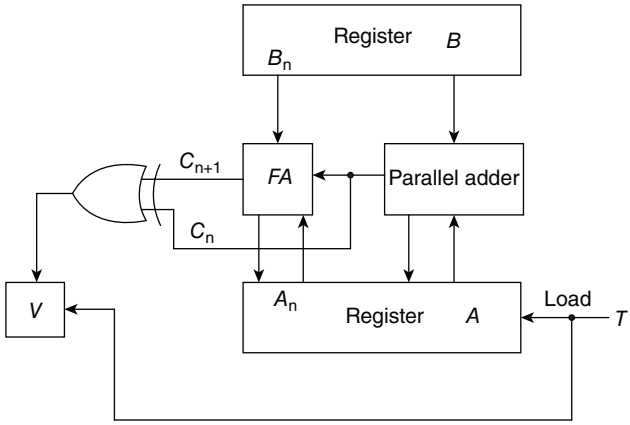


Figure 8.10 Addition of sign-2's-complement numbers

out of the full-adder is C_{n+1}. The exclusive-OR of these two carries is applied to an overflow flip-flop *V*. If, after the addition, $V = 0$, then the sum loaded into *A* is correct. If $V = 1$, there is an overflow and the *n*-bit sum is incorrect. The circuit shown in Fig. 8-10 can be specified by the following statement:

$$T: A \leftarrow A + B, \qquad V \leftarrow C_n + C_{n+1}$$

The variables in the statement are defined in Fig. 8-10. Note that variables C_n and C_{n+1} do not represent registers; they represent output carries from a parallel adder.

8.7 Arithmetic Shifts

An arithmetic shift is a microoperation that shifts a *signed* binary number to the left or right. An arithmetic shift-left multiplies a signed binary number by 2. An arithmetic shift-right divides the number by 2. Arithmetic shifts must leave the sign unchanged because the sign of the number remains the same when it is multiplied or divided by 2.

The leftmost bit in a register holds the sign bit, and the remaining bits hold the number. Figure 8-11 shows a register of *n* bits. Bit A_n in the leftmost position holds the sign bit and is designated by *A*(*S*). The number bits are stored in the portion of the register designated by *A*(*N*). A_1 refers to the least significant bit, A_{n-1} refers to the most significant position in the number bits, and *A* refers to the entire register.

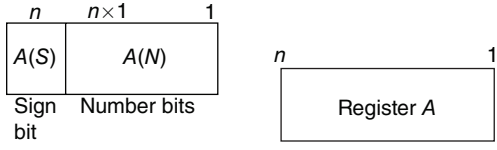


Figure 8.11 Defining register *A* for arithmetic shifts

Binary fixed-point numbers can be represented in three different ways. The manner of shifting the number stored in a register is different for each representation.

Consider first the arithmetic shift-right that divides a number by 2. This can be symbolized by one of the following statements:

$A(N) \leftarrow \text{shr } A(N),$	$A_{n-1} \leftarrow 0$	for sign-magnitude
$A \leftarrow \text{shr } A,$	$A(A) \leftarrow A(S)$	for sign-1's or sign-2's complement

In the sign-magnitude representation, the arithmetic shift-right requires a shift of the number bits with 0 inserted into the most significant position. The sign bit is not affected. In the sign-1's- or 2's-complement representation, the entire register is shifted while the sign bit remains unaltered. This is because a positive number must insert a 0 in the most significant position but a negative number must insert a 1. The following numerical examples demonstrate the procedure.

Positive number	+12:	0 01100	+6:	0 00110
Sign-magnitude	–12:	1 01100	–6:	1 00110
Sign-1's complement	–12:	1 10011	–6:	1 11001
Sign-2's complement	–12:	1 10100	–6:	1 11010

In each case the arithmetic shift-right of 12 produces a 6 without altering the sign.

For positive numbers, the result is the same in all three representations. A number in sign-magnitude, whether positive or negative, when shifted, receives a 0 in the most significant position. In the two sign-complement representations, the most significant position receives the sign bit. The latter case is sometimes called *shifting with sign extension.*

Consider now the arithmetic shift-left that multiplies a number by 2. This can be symbolized by one of the following statements:

$A(N) \leftarrow \text{shl } A(N),$	$A_1 \leftarrow 0$	for sign-magnitude
$A \leftarrow \text{shl } A,$	$A_1 \leftarrow A(S)$	for sign-1's complement
$A \leftarrow \text{shl } A,$	$A_1 \leftarrow 0$	for sign-2's complement

In the sign-magnitude representation, the number bits are shifted left with 0 inserted in the least significant position. In the sign-l's complement, the entire register is shifted and the sign bit is inserted into the least significant position. The sign-2's complement is similar, except that 0 is shifted into the least significant position. Consider the number 12 shifted to the left to produce 24:

Positive number	0 01100	0 11000
Sign-magnitude	1 01100	1 11000
Sign-1's complement	1 10011	1 00111
Sign-2's complement	1 10100	1 01000

A number shifted to the left may cause an overflow to occur. An overflow will occur after the shift if the following condition exists *before* the shift:

$A_{n-1} = 1$ for sign-magnitude

$A_n \oplus A_{n-1} = 1$ for sign-1's or sign-2's complement

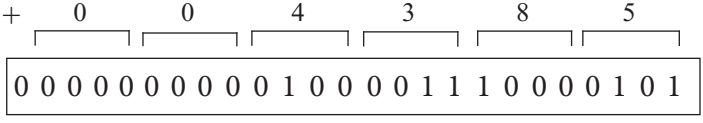
In the sign-magnitude case, a 1 in the most significant position will be shifted out and lost. In the sign-complement case, overflow will occur if the sign bit $A_n = A(S)$ is not the same as the most significant bit. Consider the following numerical example with sign-1's-complement numbers:

Initial value	9:	0 1001	Initial value	– 9: 1 0110
shift-left	– 2:	1 0010	Shift-left	+2: 0 1101

The shift-left should produce 18, but since the original sign is lost, we obtain an incorrect result with a sign reversal. If the sign bit after the shift is not the same as the sign bit before the shift, an overflow occurs. The correct result would be a number of $n + 1$ bits, with the $(n + 1)$th bit position containing the original sign of the number which is lost after the shift.

8.8 Decimal Data

The representation of decimal numbers in registers is a function of the binary code used to represent a decimal digit. A four-bit decimal code, for example, requires four flip-flops for each decimal digit. The representation of +4385 in BCD requires at least 17 flip-flops: one flip-flop for the sign and four for each digit. This number is represented in a register with 25 flip-flops as follows:



By representing numbers in decimal, we waste a considerable amount of storage space since the number of flip-flops needed to store a decimal number in a binary code is greater than the number of flip-flops needed for its equivalent binary representation. Also, the circuits required to perform decimal arithmetic are much more complex. However, there are some advantages in the use of decimal representation, mostly because computer input and output data are generated by people that always use the decimal system. A computer that uses binary representation for arithmetic operations requires data conversion from decimal to binary prior to performing calculations. Binary results must be converted back to decimal for output. This procedure is time consuming; it is worth using provided the amount of arithmetic operations is large, as is the case with scientific applications. Some applications, such as business data processing, require small amounts of arithmetic calculations. For this reason, some computers perform arithmetic calculations directly on decimal data (in binary code) and thus eliminate the need for conversion to binary and back to decimal. Large-scale computer systems usually have hardware for performing arithmetic calculations both in binary and in decimal representation. The user can specify by programmed instructions whether the computer is to perform calculations on binary or decimal data. A decimal adder was introduced in Section 5-3.

There are three ways to represent negative fixed-point decimal numbers. They are similar to the three representations of a negative binary number except for the radix change:

1. Sign-magnitude.
2. Sign-9's complement.
3. Sign-10's complement.

For all three representations, a positive decimal number is represented by 0 (for plus) followed by the magnitude of the number. It is in regard to negative numbers that the representations differ. The sign of a negative number is represented by 1, and the magnitude of the number is positive in sign-magnitude representation. In the other two representations, the magnitude is represented by the 9's or 10's complement.

The sign of a decimal number is sometimes taken as a 4-bit quantity to conform with the 4-bit representation of digits. It is customary to represent a plus with four O's and a minus with the BCD equivalent of 9, i.e., 1001. In this way all procedures developed for sign-2's-complement numbers apply also to the sign-10's-complement numbers. The addition is done by adding all digits, including the sign digit, and discarding the end carry. For example, +375 + (− 240) is done with sign-10's-complement representation as follows:

$$\begin{array}{rr} & 0\ 375 \\ + & \\ \hline & 9\ 760 \\ & 0\ 135 \end{array}$$

The 9 in the second number represents a minus, and 760 is the 10's complement of 240. An overflow is detected in this representation from the exclusive-OR of the carries into and out of the sign digits position.

Decimal arithmetic operations may employ the same symbols as binary operations provided the base of the numbers is understood to be 10 instead of 2. The statement:

$$A \leftarrow A + \overline{B} + 1$$

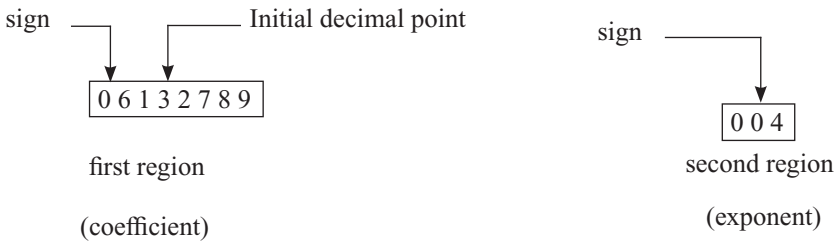
can be used to express the addition of the decimal number stored in register A to the 10's complement of the decimal number in register B. $\overline{B}$ in this case denotes the 9's complement of the decimal number. The arithmetic shifts are also applicable to decimal numbers, except that a shift-left corresponds to a multiplication by 10 and a shift-right to a division by 10. The sign-9's complement is similar to the sign-1's complement, and the sign-magnitude representation in both radix representations have similar arithmetic procedures.

If the adoption of similar symbols for binary and decimal operations were not acceptable, it would be necessary to formulate different symbols for the operations with decimal data. Sometimes the register-transfer operations are used to simulate the system by means of a computer program. In that case the two types of data can be specified by declaration statements as is done in programming languages.

8.9 Floating-point Data

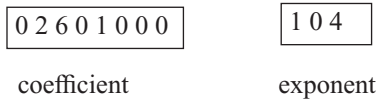
Floating-point representation of numbers needs two registers. The first represents a signed fixed-point number and the second, the position of the radix point. For example, the representation of

the decimal number +6132.789 is as follows:

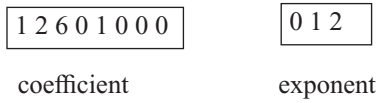


The first register has a 0 in the most significant flip-flop position to denote a plus. The magnitude of the number is stored in a binary code in 28 flip-flops, with each decimal digit occupying 4 flip-flops. The number in the first register is considered a fraction, so the decimal point in the first register is fixed at the left of the most significant digit. The second register contains the decimal number +4 (in binary code) to indicate that the *actual* position of the decimal point is four decimal positions to the right. This representation is equivalent to the number expressed by a fraction times 10 to an exponent, i.e., +6132.789 is represented as $+.6132789 \times 10^{+4}$. Because of this analogy, the contents of the first register are called the *coefficient* (and sometimes *mantissa* or *fractional part*) and the contents of the second register are called the *exponent* (or *characteristic*).

The position of the actual decimal point may be outside the range of digits of the coefficient register. For example, assuming sign-magnitude representation, the following contents:



represent the number $+.2601000 \times 10^{-4} = + .000026010000$, which produces four more 0's on the left. On the other hand, the following contents:



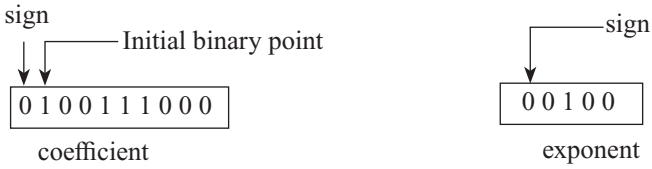
represent the number $-.2601000 \times 10^{12} = - 260100000000$, which produces five more 0's on the right.

In these examples, we have assumed that the coefficient is a fixed-point fraction. Some computers assume it to be an integer, so the initial decimal point in the coefficient register is to the right of the least significant digit.

Another arrangement used for the exponent is to remove its sign bit altogether and consider the exponent as being "biased." For example, numbers between 10^{+49} and 10^{-50} can be represented with an exponent of two digits (without sign bit) and a bias of 50. The exponent register always contains the number $E + 50$, where E is the actual exponent. The subtraction of 50 from the contents of the register gives the desired exponent. This way, positive exponents are represented in the register in the range of numbers from 50 to 99. The subtraction of 50 gives the positive values from 00 to 49. Negative exponents are represented in the register in the range of 00 to 49. The

subtraction of 50 gives the negative values in the range of −50 to −1.

A floating-point binary number is similarly represented with two registers, one to store the coefficient and the other, the exponent. For example, the number + 1001.110 can be represented as follows:

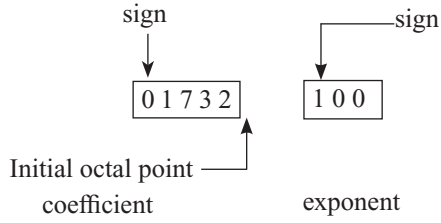


The coefficient register has ten flip-flops: one for sign and nine for magnitude. Assuming that the coefficient is a fixed-point fraction, the actual binary point is four positions to the right, so the exponent has the binary value +4. The number is represented in binary as $.100111000 \times 10^{100}$ (remember that 10^{100} in binary is equivalent to decimal 2^4).

Floating-point is always interpreted to represent a number in the following form:

$$c \cdot r^e$$

where c represents the contents of the coefficient register and e, the contents of the exponent register. The radix (base) r and the radix-point position in the coefficient are always assumed. Consider, for example, a computer that assumes integer representation for the coefficient and base 8 for the exponent. The octal number $+ 17.32 = + 1732 \times 8^{-2}$ will look like this:



When the octal representation is converted to binary, the binary value of the registers becomes:

A floating-point number is said to be *normalized* if the most significant position of the coefficient contains a nonzero digit. In this way, the coefficient has no leading zeros and contains the maximum possible number of significant digits. Consider, for example, a coefficient register that can accomodate five decimal digits and a sign. The number $+.00357 \times 10^3 = 3.57$ is not normalized because it has two leading zeros and the unnormalized coefficient is accurate to three significant digits. The number can be normalized by shifting the coefficient two positions to the left and decreasing the exponent by 2 to obtain: $+.35700 \times 10^1 = 3.5700$, which is accurate to five significant digits.

Arithmetic operations with floating-point number representation are more complicated than arithmetic operations with fixed-point numbers and their execution takes longer and requires

more complex hardware. However, floating-point representation is more convenient because of the scaling problems involved with fixed-point operations. Many computers have a built-in capability to perform floating-point arithmetic operations. Those that do not have this hardware are usually programmed to operate in this mode.

Adding or subtracting two numbers in floating-point representation requires first an alignment of the radix point, since the exponent part must be made equal before the coefficients are added or subtracted. This alignment is done by shifting one coefficient while its exponent is adjusted until it is equal to the other exponent. Floating-point multiplication or division requires no alignment of the radix point. The product can be formed by multiplying the two coefficients and adding the two exponents. Division is accomplished from the division with the coefficients and the subtraction of the divisior exponent from the exponent of the dividend.

8.10 Nonnumeric Data

The types of data considered thus far represent numbers that a computer uses as operands for arithmetic operations. However, a computer is not a machine that only stores numbers and does high-speed arithmetic. Very often, a computer manipulates symbols rather than numbers. Most programs written by computer users are in the form of characters, i.e., a set of symbols comprised of letters, digits, and various special characters. A computer is capable of accepting characters (in a binary code), storing them in memory, and performing operations on and transferring characters to an output device. A computer can function as a character-string-manipulating machine. By a *character string* is meant a finite sequence of characters written one after another.

Characters are represented in computer registers by a binary code. In Table 1-5, we listed three different character codes in common use. Each member of the code represents one character and consists of either six, seven, or eight bits, depending on the code. The number of characters that can be stored in one register depends on the length of the register and the number of bits used in the code. For example, a computer with a word length of 36 bits that uses a character code of 6 bits can store six characters per word. Character strings are stored in memory in consecutive locations. The first character in the string can be specified from the address of the first word. The last character of the string may be found from the address of the last word, or by specifying a character count, or by a special mark designating end of character string. The manipulation of characters is done in the registers of the processor unit, with each character representing a unit of information.

Various other symbols can be stored in computer registers in binary-coded form. A binary code can be adopted to represent musical notes for the production of music by computer. Special binary codes are needed to represent speech patterns for an automatic speech-recognition system. The display of characters through a dot matrix in a CRT (cathode-ray tube) screen requires a binary-coded representation for each symbol that is displayed. Status information to supervise the operation of a controlled process or a power-distribution system uses predetermined binary-coded information. The chess board and pieces to run a chess game by computer require some form of binary-coded information representation.

The operations mostly done on nonnumerical data are transfers, logic, shifts, and control decisions. The transfer operations can prepare the binary-coded information in some required order in memory and transfer the information to and from external units. Logic and shift operations provide a capability to perform data manipulation tasks that help in the decision-making process.

Logic microoperations are very useful for manipulating individual bits stored in a register or a group of bits that comprise a given binary-coded symbol. Logic operations can change bit

values, delete a group of bits, or insert new bit values into a register. The following examples show how the bits of one register are manipulated by logic and shift microoperations as a function of logic operands that are prestored in memory.

The OR microoperation can be used to set a bit or a selected group of bits in a register. The Boolean relations $x + 1 = 1$ and $x + 0 = x$ dictate that the binary variable x, when ORed with a 1, produces a 1 regardless of the binary value of x; but, when ORed with a 0, it does not change the value of x. By ORing a given bit A_1 in a register with a 1, it is possible to set bit A_1 to 1 regardless of its previous value. Consider the following specific example:

0101 0101	A
1111 0000	B
1111 0101	$A \leftarrow A \vee B$

The logic operand in B has 1's in the four high-order bit positions. By ORing this value with the present value of A, it is possible to set the four high-order bits of A to 1's but leave the four low-order bits unchanged. Thus, the OR microoperation can be used to selectively set bits of a register.

The AND microoperation can be used to clear a bit or a selected group of bits in a register. The Boolean relationships $x \cdot 0 = 0$ and $x \cdot 1 = x$ dictate that binary variable x, when ANDed with a 0, produces a 0 regardless of the binary value of x; but, when ANDed with a 1, it does not change the value of x. A given bit A_i in register A can be cleared to 0 if it is ANDed with a 0. Consider a logic operand $B = 0000\ 1111$. When this operand is ANDed with the contents of a register, it will clear the four high-order bits of the register but leave the four low-order bits unchanged:

0101 0101	A
0000 1111	B
0000 0101	$A \leftarrow A \wedge B$

The AND microoperation can be used to selectively clear bits of a register. The AND operation is sometimes called a *mask* operation because it masks or removes all l's from a selected portion of a register.

The AND operation followed by an OR operation can be used to change a bit or a group of bits from a given value to a new desired value. This is done by first masking the bits in question and then ORing the new value. For example, suppose an A register contains eight bits, 0110 0101. To replace the four high-order bits by the value 1100, we first mask the four unwanted bits:

0110 0101	A
0000 1111	$B1$
0000 0101	$A \leftarrow A \wedge B1$

and then insert the new value:

0000 0101	A
1100 0000	$B2$
1100 0101	$A \leftarrow A \wedge B1$

The mask operation is an AND microoperation and the insert operation is an OR microoperation.

The XOR (exclusive-OR) microoperation can be used to complement a bit or a selected group of bits in a register. The Boolean relationships $x \oplus 1 = x'$ and $x \oplus 0 = x$ dictate that the binary variable x be complemented when XORed with a 1 and remain unchanged otherwise. By XORing a given bit in a register with a 1, it is possible to complement the selected bit. Consider the numerical example:

1101 0101	A
1111 0000	B
0010 0101	$A \leftarrow A \oplus B$

The four high-order bits of A are complemented after the exclusive-OR operation with operand B. The XOR microoperation can be used to selectively complement bits of a register. If the operand B has all 1's, the XOR operation will complement all the bits of A. If the contents of A are XORed with their own value, the register will be cleared, since $x \oplus x = 0$:

0101 0101	A
0101 0101	B
0000 0000	$A \leftarrow A \oplus A$

The value of individual bits in a register can be determined by first masking all bits, except the one in question, and then checking if the register is equal to 0 Suppose we want to determine if bit 4 in register A is 0 or 1:

101 x 010	A
0001000	B
000 x 000	$A \leftarrow B \wedge A$

The bit marked x can be 0 or 1. When all other bits are masked with the operand in B, register A will contain all 0's if bit 4 was a 0. If bit 4 was originally a 1, this bit will remain a 1. By checking if the contents of A are 0 or not, we determine whether bit 4 was 0 or 1.

If each bit in a register must be checked for 0 or 1, it is more convenient to shift the register to the left and transfer the high-order bit into a special 1-bit register which is usually called the carry flip-flop. After every shift, the carry can be checked for 0 or 1 and a decision made depending on the outcome.

Shift operations are useful for packing and unpacking binary-coded information. Packing of binary-coded information such as characters is an operation that groups two or more characters in one word. Unpacking is a reverse operation that separates two or more characters stored in one word into individual characters. Consider the packing of BCD digits that are first entered as ASCII characters. The ASCII character code for digits 5 and 9 are obtained from Table 1-5. Each contains seven bits and a 0 is inserted in the high-order position as shown below. The character 5 is transferred to register A, and 9 to register B. The four high-order bits are of no use for BCD representation, so they are masked out. The packing of the two BCD digits into register A consists of shifting register A four times to the left (with 0's inserted in the low-order bit positions) and then ORing the contents of the registers;

	A	*B*
ASCII 5 =	0011 0101	0011 1001 = ASCII 9
AND with 0000 1111	0000 0101	0000 1001
Shift *A* left four times	0101 0000	
$A \leftarrow A \vee B$	0101 1001 = BCD 59	

A shift operation with 0 inserted in the extreme bit is considered a logical shift microoperation.

The binary information available in a register during logical operations is called a *logical word.* A logical word is interpreted to be a bit string as opposed to character strings or numerical data. Each bit in a logical word functions in exactly the same way as every other bit; in other words, the unit of information of a logical word is a bit.

8.11 Instruction Codes

The internal organization of a digital system is defined by the registers it employs and the sequence of microoperations it performs on data stored in the registers. In a *special-purpose* digital system, the sequence of microoperations is fixed and the system performs the same specific task over and over again. A digital computer is a *general-purpose* digital system capable of executing various operations and, in addition, can be instructed as to what specific sequence of operations it must perform. The user of a computer can control the process by means of *a. program,* i.e., a set of instructions that specify the operations, operands, and the sequence in which processing has to occur. The data-processing task may be altered simply by specifying a new program with different instructions or by specifying the same instructions with different data. Instruction codes, together with data, are stored in memory. The control reads each instruction from memory and places it in a control register. The control then interprets the instruction and proceeds to execute it by issuing a sequence of control functions. Every general-purpose computer has its own unique instruction repertoire. The ability to store and execute instructions, the stored-program concept, is the most important property of a general-purpose computer.

An *instruction code* is a group of bits that tell the computer to perform a specific operation. It is usually divided into parts, each having its own particular interpretation. The most basic part of an instruction code is its operation part. The *operation code* of an instruction is a group of bits that define an operation such as add, subtract, multiply, shift, and complement. The set of machine operations formulated for a computer depends on the processing it is intended to carry out. The total number of operations thus obtained determines the set of machine operations. The number of bits required for the operation part of the instruction code is a function of the total number of operations used. It must consist of at least n bits for a given 2^n (or less) distinct operations. The designer assigns a bit combination (a code) to each operation. The control unit of the computer is designed to accept this bit configuration at the proper time in a sequence and to supply the proper command signals to the required destinations in order to execute the specified operation. As a specific example, consider a computer using 32 distinct operations, one of them being an ADD operation. The operation code may consist of five bits, with a bit configuration 10010 assigned to the ADD operation.

When the operation code 10010 is detected by the control unit, a command signal is applied to an adder circuit to add two numbers.

The operation part of an instruction code specifies the operation to be performed. This operation must be executed on some data, usually stored in computer registers. An instruction code, therefore, must specify not only the operation but also the registers where the operands are to be found as well as the register where the result is to be stored. These registers may be specified in an instruction code in two ways. A register is said to be specified *explicitly* if the instruction code contains special bits for its identification. For example, an instruction may contain not only an operation part but also a memory address. We say that the memory address specifies explicitly a memory register. On the other hand, a register is said to be specified *implicitly* if it is included as part of the definition of the operation, i.e., if the register is implied by the operation part of the code.

8.11.1 Instruction-Code Formats

The format of an instruction is usually depicted in a rectangular box symbolizing the bits of the instruction as they appear in memory words or in a control register. The bits of the instruction are sometimes divided into groups that subdivide the instruction into parts. Each group is assigned a symbolic name, such as *operation-code* part or *address* part. The various parts specify different functions for the instruction, and when shown together they constitute the instruction-code format.

Consider, for example, the three instruction-code formats specified in Fig. 8-12. The instruction format in (a) consists of an operation code which implies a register in the processor unit. It can be used to specify operations such as "clear a processor register," or "complement a register," or "transfer the contents of one register to a second register." The instruction format in (b) has an operation code followed by an operand. This is called an *immediate* operand instruction because the operand follows immediately after the operation-code part of the instruction. It can be used to specify operations such as "add the operand to the present contents of a register" or "transfer the operand to a processor register," or it can specify any other operation to be done between the contents of a register and the given operand. The instruction format specified in Fig. 8-12(c) is similar to the one in (b) except that the operand must be extracted from memory at the location specified by the address part of the instruction. In other words, the operation specified by the operation code is done between a processor register and an operand which can be stored in memory anywhere. The address of this operand in memory is included in the instruction.

Let us assume that we have a memory unit with 8 bits per word and that an operation code contains 8 bits. The placement of the three instruction codes in memory is depicted in Fig. 8-13. At address 25, we have an implied instruction that specifies an operation: "transfer the contents of processor register *R* into processor register *A*." This operation can be symbolized by the statement:

$$A \leftarrow R$$

In memory addresses 35 and 36, we have an immediate operand instruction that occupies two words. The first word at address 35 is the operation code for the instruction, "transfer the operand to register *A*," symbolized as:

$$A \leftarrow \text{operand}$$

The operand itself is stored immediately after the operation code at address 36.

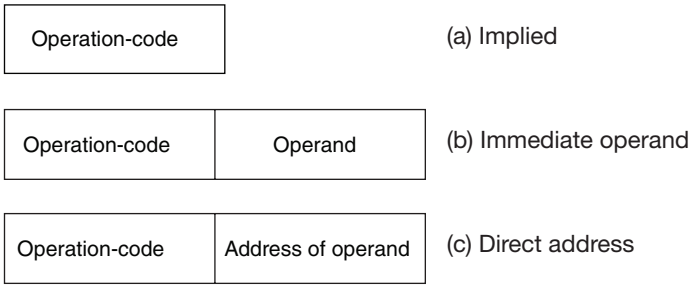


Figure 8.12 Three possible instruction formats

In addresses 45 and 46, there is a direct address instruction that specifies the operation:

$$A \leftarrow M[\text{address}]$$

This symbolizes a memory-transfer operation of an operand which is specified by the address part of the instruction. The second word of the instruction at address 46 contains the *address*, and its value is binary 70. Therefore, the operand to be transferred to register A is the one stored in address 70, and its value is shown to be binary 28. Note that the instruction is stored in memory at some address. This instruction has an address part which gives the address of the operand. To avoid the confusion of saying the word "address" so many times, it is customary to refer to a memory address as a "location." Thus the direct address instruction is stored in locations 45 and 46. The address of the operand is in location 46, and the operand is available from location 70.

It must be realized that the placing of instructions in memory as shown in Fig, 8-13 is only one of many possible alternatives. Only very small computers have 8-bit words. Large computers may have from 16 to 64 bits per word. In most computers, the entire instruction can be placed in one word, and in some, even two or more instructions can be placed in a single memory word.

The instruction formats shown in Fig. 8-12 are three of many possible formats that can be formulated for digital computers. They are presented here just as an example and should not be considered the only possibilities. Subsequent chapters, especially Chapters 11 and 12, present and discuss other instructions and instruction-code formats.

At this point we must recognize the relationship between an *operation* and a *microoperation* as applied to a digital computer. An operation is specified by an instruction stored in computer memory. It is a binary code that tells the computer to perform a specific operation. The control unit receives the instruction from memory and interprets the operation-code bits. It then issues a sequence of control functions that perform microoperations in internal computer registers.

For every instruction in memory that specifies an operation, the control issues a sequence of microoperations which are needed for the hardware implementation of the specified operation code. An operation is specified by a user in the form of an instruction to the computer. A microoperation is an elementary operation which is constrained by the available hardware inside the computer.

8.11.2 Macrooperations versus Microoperations

There are occasions when it is convenient to express a sequence of microoperations with a single statement. A statement that requires a sequence of microoperations for its implementation is

called a *macrooperation.* A statement in the register-transfer method of notation that defines an instruction is a *macrooperation* statement, although macrooperation statements can be used on other occasions as well. The register-transfer method can be used to define the operation specified by a computer instruction because all instructions specify some register-transfer operation which must be executed by computer hardware.

One cannot tell from looking at an isolated register-transfer statement whether it represents a macro- or microoperation, since both types of statements denote some register-transfer statement. The only way to distinguish between them is to recognize from the context and the internal hardware of the system in question whether the statement is executed with one control function or not. If the statement can be executed with a single control function, it represents a microoperation. If the hardware execution of the statement requires two or more control functions, the statement is taken to be a macrooperation. Only if one knows the hardware constraints of the system can this question be answered.

Consider, for example, the instruction in Fig. 8-13 symbolized by the statement:

$$A \leftarrow \text{operand}$$

This statement is a macrooperation because it specifies a computer instruction. To execute the instruction, the control unit must issue control functions for the following sequence of microoperations:

1. Read the operation code from address 35.
2. Transfer the operation code to a control register.
3. The control decodes the operation code and recognizes it as an immediate operand instruction, so it reads the operand from address 36.
4. The operand read from memory is transferred into register *A*.

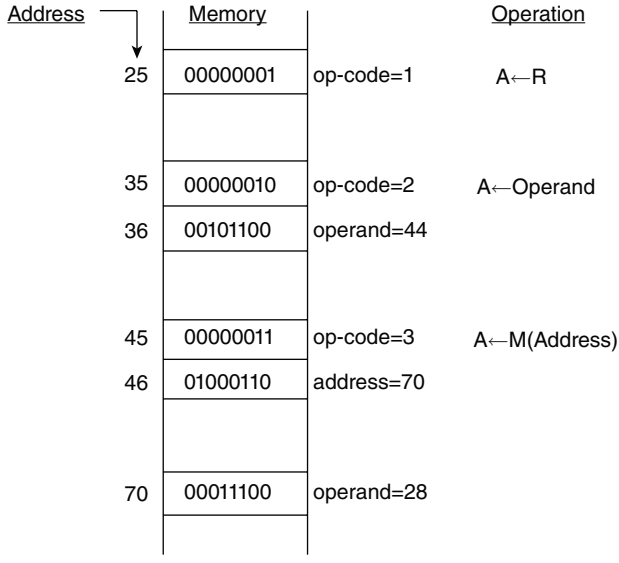


Figure 8.13 Memory representation of instructions